Harness Continuous Delivery Advanced Field Guide

Barriers, looping and matrix strategies, Datadog continuous verification, multi-service deployments, ServiceNow approvals, Kubernetes rollbacks, and service variables

Service, environment and infrastructure overrides
Expressions, runtime inputs and input sets
Flow control: barriers and resource constraints
Multi-service and multi-environment orchestration
Change management with ServiceNow
Verification with Datadog
Rollback design for Kubernetes

Edition: October 2026 | Audience: platform engineers, release engineers, SREs
YAML in this guide targets the NextGen (v0) pipeline schema unless stated otherwise.

Contents

1. About This Guide and the Harness CD Model 6

1.1 Purpose and scope 6

1.2 Reading conventions 6

1.3 The CD object model 6

1.4 Pipeline structure refresher 8

1.5 The expression engine in one page 9

1.6 A running example used throughout 9

1.7 Chapter map 10

2. Services and Service Variables in Depth 11

2.1 Why service design matters 11

2.2 Anatomy of a Kubernetes service 11

2.3 Service variables 12

2.4 Artifact sources and primary artifact selection 14

2.5 Manifest handling 15

2.6 Service hooks 16

2.7 Config files 16

2.8 Service-level checklist 16

2.9 Common mistakes with service variables 17

3. Environments, Infrastructure, and the Override Hierarchy 18

3.1 Environments as first-class objects 18

3.2 Infrastructure definitions 18

3.3 Environment variables 19

3.4 Overrides: the V2 model 19

3.5 Environment groups 21

3.6 Infrastructure selection at run time 21

3.7 Cluster identity and safety 21

3.8 What belongs where: a decision table 22

4. Variables, Expressions, Runtime Inputs, and Input Sets 23

4.1 Variable scopes 23

4.2 Fixed, runtime, and expression values 23

4.3 Expressions beyond substitution 24

4.4 Secrets in expressions 26

4.5 Input sets 26

4.6 Pipeline variables vs input sets vs service variables 27

4.7 Debugging expressions 27

5. Kubernetes Deployment Strategies and Step Reference 29

5.1 How Harness deploys to Kubernetes 29

5.2 Rolling deployment 29

5.3 Canary deployment 30

5.4 Blue-green deployment 31

5.5 Other Kubernetes steps you will use 32

5.6 Choosing a strategy 33

5.7 Timeouts and steady state 33

6. Barriers and Resource Constraints 34

6.1 What a barrier is 34

6.2 Defining and using barriers 34

6.3 Barrier rules and constraints 36

6.4 Barriers across multiple services and regions 37
6.5 Designing with barriers: a checklist 38
6.6 Resource constraints 38
6.7 Barrier vs resource constraint vs approval 39
6.8 Troubleshooting barriers 39

7. Looping Strategies: Matrix, Repeat, and Parallelism 40

7.1 Why loop 40
7.2 Matrix 40
7.3 Repeat 42
7.4 Parallelism 43
7.5 Strategy expression reference 43
7.6 Looping on steps and step groups 43
7.7 Failure behavior in loops 44
7.8 Combining matrix with services and environments 44
7.9 Matrix versus multi-service 45
7.10 Practical limits and good behavior 45
7.11 Worked example: three-wave production rollout 45

8. Multi-Service and Multi-Environment Deployments 47

8.1 The problem space 47
8.2 Multiple services in one stage 47
8.3 Multiple environments in one stage 49
8.4 Multi-service by multi-environment 49
8.5 Filters and environment groups 50
8.6 Expressions in multi-service stages 50
8.7 Services with different shapes in one stage 51
8.8 Multi-service with approvals and verification 51
8.9 Rollback semantics in multi-deployment stages 51
8.10 Service propagation across stages 52
8.11 Environment propagation and chained pipelines 52
8.12 Performance and capacity 53
8.13 Checklist for multi-service releases 53

9. Approvals and ServiceNow Integration 54

9.1 Approval types in Harness CD 54
9.2 Harness native approval 54
9.3 Why ServiceNow 55
9.4 ServiceNow connector 55
9.5 Creating a change request 56
9.6 Approving on a ticket 57
9.7 Updating tickets 58
9.8 Import sets 59
9.9 End-to-end change-managed stage 60
9.10 Approval timeouts, rejection, and expiry 62
9.11 Approval design patterns 62
9.12 Troubleshooting approvals 62

10. Continuous Verification with Datadog 63

10.1 What continuous verification does 63
10.2 Building blocks 63
10.3 Datadog connector 63
10.4 Defining the health source 64
10.5 The Verify step 65

10.6 Verification across deployment strategies 67
10.7 Failure handling for verification 67
10.8 Reading analysis results 68
10.9 Why verification returns no data 68
10.10 Cost, load, and API usage 69
10.11 Putting Datadog and rollback together 69
10.12 Verification design checklist 71

11. Rollbacks in Kubernetes 72

11.1 What rollback means in Harness 72
11.2 How Harness tracks releases 72
11.3 Rolling rollback 73
11.4 Canary rollback 74
11.5 Blue-green rollback 74
11.6 Failure strategies that trigger rollback 75
11.7 Pipeline rollback 76
11.8 Rolling back after a successful deployment 76
11.9 Rollback hazards specific to Kubernetes 78
11.10 Native kubectl rollout undo versus Harness rollback 79
11.11 Testing rollbacks 79
11.12 Rollback design checklist 79

12. Failure Strategies, Conditional Execution, and Step Groups 80

12.1 Failure strategies in depth 80
12.2 Conditional execution 81
12.3 Step groups 82
12.4 Timeouts at every level 83
12.5 Retries and idempotency 83
12.6 Execution input and pauses 84
12.7 Notifications 84
12.8 Design patterns 84

13. Governance: Templates, Policy, Freeze Windows, RBAC, and Triggers 85

13.1 Why governance belongs in the pipeline 85
13.2 Templates 85
13.3 OPA policies 87
13.4 Freeze windows 88
13.5 RBAC and resource groups 89
13.6 Triggers 89
13.7 Audit and compliance evidence 90
13.8 Secrets 90
13.9 Pipeline as code 90

14. Troubleshooting and Operations 91

14.1 A systematic approach 91
14.2 Failure classes and first checks 91
14.3 Delegate troubleshooting 91
14.4 Kubernetes-specific issues 92
14.5 Looping and multi-service problems 92
14.6 Variable and override problems 93
14.7 ServiceNow troubleshooting 93
14.8 Datadog troubleshooting 93
14.9 Operating pipelines in production 93
14.10 Incident playbook: bad release in production 94

15. Capstone: A Production-Grade Release Pipeline 95
15.1 Requirements 95
15.2 Architecture 95
15.3 Entities assumed to exist 95
15.4 The pipeline YAML 96
15.5 Walkthrough of the design decisions 101
15.6 Operational parameters per environment 102
15.7 Rehearsal plan before first production use 102

Appendix A: Quick References 103
A.1 Expression cheat sheet 103
A.2 Runtime input modifiers 103
A.3 Step type reference (Kubernetes CD) 104
A.4 Failure strategy quick table 104
A.5 Override precedence summary 104

Appendix B: Production Readiness Checklist 105
B.1 Pipeline structure 105
B.2 Safety 105
B.3 Change management 105
B.4 Verification 105
B.5 Rollback 106
B.6 Operations 106

Appendix C: Glossary 107

Appendix D: Validation and Further Reading 108
D.1 Validate this material against your account 108
D.2 Topics outside the scope of this guide 108

1. About This Guide and the Harness CD Model

1.1 Purpose and scope

This guide is a working reference for engineers who already deploy with Harness Continuous Delivery (CD) and now need to design pipelines that are safe, repeatable, and governed at scale. It does not repeat the "first pipeline" tutorial. It focuses on the capabilities that separate a demo pipeline from a production release process:

- **Service variables** and the full override hierarchy across service, environment, and infrastructure.
- **Expressions, runtime inputs, and input sets** at an advanced level, including execution-time inputs and output variables.
- **Flow control** with barriers and resource constraints.
- **Looping strategies**: matrix, repeat, and parallelism.
- **Multi-service and multi-environment deployments**, including environment groups and filters.
- **Change management** with ServiceNow create, update, and approval steps, plus Harness native approvals.
- **Continuous verification (CV)** with Datadog health sources and the Verify step.
- **Kubernetes rollbacks**: rolling, canary, blue-green, pipeline rollback, and post-deployment rollback.
- **Governance and operations**: failure strategies, conditional execution, freeze windows, OPA policy, notifications, and troubleshooting.

Every chapter ends with practical guidance: where each feature fits, what typically goes wrong, and what to check first.

> **How to use the YAML in this guide**
>
> The YAML samples use the NextGen (v0) pipeline schema, which is what the Harness UI YAML editor and the `pipeline` resource type produce. Field names evolve between releases, and some features sit behind account-level feature flags or settings. Treat each sample as a reviewed starting point. Before you commit a pipeline, paste it into the Pipeline Studio YAML editor, which validates against the live schema for your account, and run it in a non-production environment first.

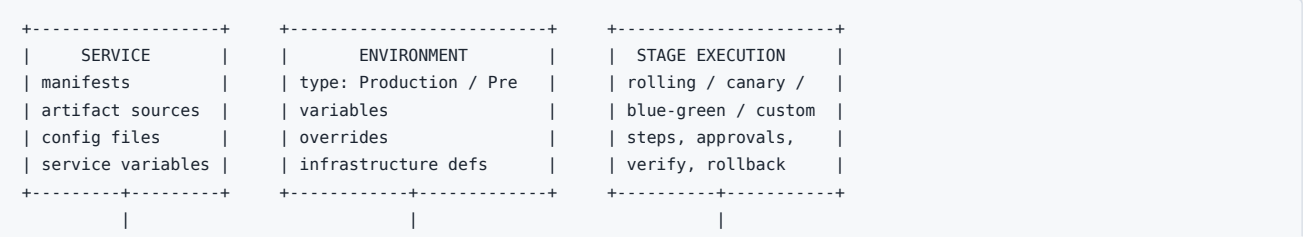
1.2 Reading conventions

Convention	Meaning
`<+expression>`	A Harness runtime expression, resolved by the platform when the step or stage executes.
`<+input>`	A runtime input. The value must be supplied when the pipeline runs or by an input set.
`<+input>.allowedValues(a,b)`	A runtime input restricted to a fixed list.
`identifier`	The immutable, URL-safe ID of an entity. Names are display labels, identifiers are references.
`Ref` suffix	A reference to another entity, for example `serviceRef`, `connectorRef`, `environmentRef`.
Scope	Account, organization, or project. A reference can point to an entity at a wider scope using `account.` or `org.` prefixes.

1.3 The CD object model

Harness CD separates three questions that other tools mash into a single script:

1. **What** am I deploying? This is the **service**.
2. **Where** am I deploying it? This is the **environment** and its **infrastructure definition**.
3. **How** do I deploy it? This is the **execution strategy** inside a deployment stage.



```
          +------------+---------------+-----------------------------+
                       v
              +---------------------+
              |  Deployment Stage   |  runs on a Delegate
              +---------------------+
```

1.3.1 Service

A service is the reusable definition of a deployable workload. For Kubernetes it typically contains:

- One or more **manifests** (Kubernetes manifests, Helm chart, Kustomize, or OpenShift templates) sourced from Git, a Helm repository, or the Harness File Store.
- One or more **artifact sources**, such as Docker Registry, ECR, GCR, Artifactory, or Nexus. One artifact is designated **primary**, others can be **sidecars**.
- **Config files**, which are plain files or secrets made available to the deployment.
- **Service variables**, named values that can be referenced as `<+serviceVariables.name>`.
- Optional **service hooks** (pre and post hooks that run around manifest fetch and rendering).

The service holds nothing about the target cluster. That separation is what makes the same service deployable to dev, staging, and production.

1.3.2 Environment and infrastructure definition

An environment is a logical target, such as `dev`, `qa`, or `prod`, typed as **PreProduction** or **Production**. The type matters for reporting, RBAC, and freeze windows.

Inside an environment you define:

- **Infrastructure definitions**: the concrete place to deploy, for example a Kubernetes cluster connector plus namespace and release name.
- **Environment variables**, referenced as `<+env.variables.name>`.
- **Overrides**: values or files that replace what the service defines, either for the whole environment or for a specific service.
- **Config files and manifests overrides** that apply only in that environment.

1.3.3 Deployment stage

A deployment stage binds a service to an environment and infrastructure, then declares an **execution** section (the forward path) and a **rollbackSteps** section (what to do on failure). Harness can scaffold the execution for a deployment type. For Kubernetes the typical scaffolds are:

Strategy	Core steps	Typical use
Rolling	`K8sRollingDeploy`	Default, in-place incremental replacement.
Canary	`K8sCanaryDeploy`, `K8sCanaryDelete`, then `K8sRollingDeploy`	Gradual exposure with manual or automated promotion.
Blue-green	`K8sBlueGreenDeploy`, `K8sBGSwapServices`	Instant traffic cutover with fast revert.
Basic apply	`K8sApply`	Apply arbitrary manifests without release tracking features.

1.3.4 Delegates

All CD work runs through a **delegate**, a lightweight worker installed in your network. The delegate pulls manifests, talks to the Kubernetes API, calls ServiceNow and Datadog, and returns results to the Harness control plane. Delegate placement determines what it can reach, so several advanced features in this guide (ServiceNow, Datadog, private clusters) depend on **delegate selectors** and network egress being right.

> **Delegate selectors on connectors and steps**
>
> Put delegate selectors on the connector for stable routing (for example, `prod-eu`), and use step-level delegate selectors only when one step needs a different network path. Tag delegates by environment tier and region so selectors stay readable.

1.4 Pipeline structure refresher

A pipeline contains stages. A stage contains steps, optionally grouped into step groups. Steps within a stage run sequentially unless you wrap them in a `parallel` block. Stages run sequentially unless you place them in a `parallel` block.

```yaml
pipeline:
  name: orders-release
  identifier: orders_release
  projectIdentifier: payments
  orgIdentifier: platform
  tags: {}
  variables:
    - name: releaseTicket
      type: String
      value: <+input>
  stages:
    - stage:
        name: deploy_dev
        identifier: deploy_dev
        type: Deployment
        spec:
          deploymentType: Kubernetes
          service:
            serviceRef: orders_api
          environment:
            environmentRef: dev
            deployToAll: false
            infrastructureDefinitions:
              - identifier: dev_cluster
          execution:
            steps:
              - step:
                  name: Rollout
                  identifier: rollout
                  type: K8sRollingDeploy
                  timeout: 10m
                  spec:
                    skipDryRun: false
                    pruningEnabled: false
            rollbackSteps:
              - step:
                  name: Rollback
                  identifier: rollback
                  type: K8sRollingRollback
                  timeout: 10m
                  spec:
                    pruningEnabled: false
        failureStrategies:
          - onFailure:
              errors:
                - AllErrors
              action:
                type: StageRollback
```

1.4.1 Stage-level sections you will touch repeatedly

Section	Purpose
`spec.service` or `spec.services`	One service or a list of services (multi-service).
`spec.environment` or `spec.environments` or `spec.environmentGroup`	One environment, many, or a group.
`spec.execution.steps`	Forward deployment path.
`spec.execution.rollbackSteps`	Rollback path.
`failureStrategies`	What the stage does when a step or the stage fails.
`when`	Conditional execution for the stage.

Section	Purpose
`strategy`	Looping: matrix, repeat, or parallelism.
`variables`	Stage variables, referenced as `<+stage.variables.name>`.

1.5 The expression engine in one page

Harness expressions are written between `<+` and `>`. They resolve against a tree of context objects. The ones you will use most in CD are:

Namespace	Examples	Notes
`pipeline`	`<+pipeline.name>`, `<+pipeline.sequenceId>`, `<+pipeline.variables.x>`	Pipeline-wide values.
`stage`	`<+stage.name>`, `<+stage.variables.x>`	Current stage.
`service`	`<+service.name>`, `<+service.identifier>`	Current service in the stage.
`serviceVariables`	`<+serviceVariables.replicas>`	Service variables after overrides are applied.
`env`	`<+env.name>`, `<+env.identifier>`, `<+env.variables.region>`	Current environment.
`infra`	`<+infra.namespace>`, `<+infra.releaseName>`, `<+infra.name>`	Current infrastructure definition.
`artifact`	`<+artifact.image>`, `<+artifact.tag>`, `<+artifact.imagePath>`	Primary artifact.
`secrets`	`<+secrets.getValue("db_password")>`	Secret reference, masked in logs.
`execution`	`<+execution.steps.stepId.output.outputVariables.x>`	Outputs of earlier steps.
`matrix`, `repeat`, `strategy`	`<+matrix.region>`, `<+repeat.item>`, `<+strategy.iteration>`	Looping context.
`approval` (via step outputs)	`<+pipeline.stages.stage.spec.execution.steps.approval.output.approvalActivities>`	Approval metadata.

Expressions are evaluated lazily, at the moment the owning step or stage runs. A few consequences matter in advanced designs:

- You cannot reference an output that has not been produced yet. A stage cannot read `execution.steps.x.output` for a step in a later stage.
- Expressions inside runtime-input fields are resolved after the input is provided.
- In looped stages, the same expression resolves differently per iteration. This is how a single stage definition produces many distinct deployments.

> **Fixed, runtime, and expression modes**
>
> Every field in the UI can be set in one of three modes: **Fixed value**, **Runtime input** (`<+input>`), or **Expression** (`<+...>`). Switching modes rewrites the YAML. If a pipeline suddenly prompts for a value you thought was fixed, check whether the field was accidentally switched to runtime input.

1.6 A running example used throughout

The following fictional product is used to keep examples consistent.

Item	Value
Product	`orders` platform
Services	`orders_api`, `orders_worker`, `orders_web`
Environments	`dev`, `qa`, `staging`, `prod_eu`, `prod_us`
Cluster per environment	One Kubernetes cluster, namespace `orders`
Observability	Datadog (metrics, APM, logs)

Item	Value
Change management	ServiceNow change requests for production
Registry	A Docker registry connector named `registry_main`

The release flow we are building toward:

```
dev (auto)  ->  qa (auto + verify)  ->  staging (verify + CAB check)
                                              |
                                              v
                              prod_eu || prod_us   (matrix, barrier-synced,
                                                    ServiceNow approved,
                                                    Datadog verified, rollback armed)
```

By the end of the guide, Chapter 15 assembles every concept into one pipeline for this flow.

1.7 Chapter map

Chapter	Topic
2	Services and service variables in depth
3	Environments, infrastructure, and the override hierarchy
4	Variables, expressions, runtime inputs, and input sets
5	Kubernetes deployment strategies and step reference
6	Barriers and resource constraints
7	Looping strategies: matrix, repeat, parallelism
8	Multi-service and multi-environment deployments
9	Approvals and ServiceNow integration
10	Continuous verification with Datadog
11	Rollbacks in Kubernetes
12	Failure strategies, conditional execution, and step groups
13	Governance: templates, policy, freeze windows, notifications
14	Troubleshooting and operations
15	Capstone pipeline
A	Quick references and checklists

2. Services and Service Variables in Depth

2.1 Why service design matters

The service is the contract between application teams and the platform. A well-designed service lets a release engineer deploy any version to any environment by changing only inputs. A poorly designed one hard-codes cluster names, namespaces, replica counts, and image tags into manifests, which forces copy-pasted pipelines.

Good service design follows three rules:

1. **No environment knowledge in the service.** Anything that differs between environments is a variable, an override, or an expression.
2. **Artifacts are inputs, not constants.** The primary artifact tag is almost always a runtime input or an expression coming from CI.
3. **Manifests are templates.** They consume values through `<+...>` expressions or Helm and Go templating, never literal values that vary.

2.2 Anatomy of a Kubernetes service

```
service:
  name: orders_api
  identifier: orders_api
  orgIdentifier: platform
  projectIdentifier: payments
  serviceDefinition:
    type: Kubernetes
    spec:
      variables:
        - name: replicas
          type: Number
          value: 2
        - name: logLevel
          type: String
          value: info
        - name: dbPassword
          type: Secret
          value: account.orders_db_password
      manifests:
        - manifest:
            identifier: k8s_manifests
            type: K8sManifest
            spec:
              store:
                type: Github
                spec:
                  connectorRef: github_main
                  gitFetchType: Branch
                  paths:
                    - deploy/k8s
                  repoName: orders-api
                  branch: main
              valuesPaths:
                - deploy/values/common.yaml
              skipResourceVersioning: false
              enableDeclarativeRollback: false
      artifacts:
        primary:
          primaryArtifactRef: <+input>
          sources:
            - spec:
                connectorRef: registry_main
                imagePath: acme/orders-api
                tag: <+input>
                digest: ""
              identifier: main_image
```

```
            type: DockerRegistry
      configFiles:
        - configFile:
            identifier: app_properties
            spec:
              store:
                type: Harness
                spec:
                  files:
                    - /orders/app.properties
  gitOpsEnabled: false
```

2.2.1 Key points in the definition

- `variables` define the service variable names, types, and default values. These are the values downstream stages and overrides can replace.
- `valuesPaths` on a `K8sManifest` supplies values files that are merged and exposed to Go templates in your manifests.
- `skipResourceVersioning: false` means Harness versions ConfigMaps and Secrets (see Chapter 11), which has direct rollback implications.
- `primaryArtifactRef: <+input>` with multiple `sources` lets the operator choose which source to deploy at run time.

2.3 Service variables

2.3.1 Types

Type	Description	Example
`String`	Free text.	`logLevel: info`
`Number`	Numeric; usable in arithmetic expressions.	`replicas: 2`
`Secret`	Reference to a Harness secret, masked in logs and never printed.	`account.orders_db_password`

Each variable can be set as fixed, runtime input, or expression. A variable declared with `value: <+input>` forces the person or input set running the pipeline to supply it.

2.3.2 Referencing service variables

Inside the pipeline:

```
<+serviceVariables.replicas>
<+serviceVariables.logLevel>
<+secrets.getValue(<+serviceVariables.dbPassword>)>
```

The `serviceVariables` namespace returns the **effective** value, meaning the service default after environment and infrastructure overrides were applied. Use `<+service.variables.name>` only when you want the value defined on the service itself, regardless of overrides, which is rarely what you want in a deployment step.

> **serviceVariables vs service.variables**
>
> `<+serviceVariables.x>` is the resolved value after overrides. `<+service.variables.x>` is the value as defined on the service. If you build manifests, you almost always want `<+serviceVariables.x>`.

2.3.3 Using service variables inside manifests

In a plain Kubernetes manifest, an expression can appear anywhere a value is allowed:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: orders-api
  namespace: <+infra.namespace>
spec:
```

```
  replicas: <+serviceVariables.replicas>
  selector:
    matchLabels:
      app: orders-api
  template:
    metadata:
      labels:
        app: orders-api
        release: <+env.name>
    spec:
      containers:
        - name: api
          image: <+artifact.image>
          env:
            - name: LOG_LEVEL
              value: "<+serviceVariables.logLevel>"
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: orders-db
                  key: password
```

Prefer `secretKeyRef` for secrets that live in Kubernetes. If a value must flow from a Harness secret into a manifest, remember that rendering a Harness secret into a ConfigMap or a plain env value stores it in cluster state in the clear. Harness masks it in logs, not in the cluster.

2.3.4 Using service variables with Helm

For Helm charts, service variables are typically mapped into values through a values file template or through the `--set` style value overrides in the manifest spec. A common pattern is a values file stored in Git that contains Harness expressions:

```
# deploy/values/common.yaml
replicaCount: <+serviceVariables.replicas>
image:
  repository: <+artifact.imagePath>
  tag: <+artifact.tag>
logLevel: <+serviceVariables.logLevel>
ingress:
  host: orders-<+env.variables.region>.example.com
```

Harness evaluates the expressions in the values file before passing the file to Helm. This keeps the chart generic while the values stay environment-aware.

2.3.5 Using service variables with Kustomize

With Kustomize, put expressions in patch files. A patch such as `patches/replicas.yaml` can contain `replicas: <+serviceVariables.replicas>`. Harness resolves expressions in the fetched files before invoking `kustomize build`.

2.3.6 Service variables in scripts and steps

Shell Script steps can read service variables directly:

```
- step:
    type: ShellScript
    name: Print Config
    identifier: print_config
    spec:
      shell: Bash
      source:
        type: Inline
        spec:
          script: |
            echo "Deploying <+service.name> with log level <+serviceVariables.logLevel>"
            echo "Replicas: <+serviceVariables.replicas>"
      environmentVariables: []
```

```
      outputVariables: []
    timeout: 5m
```

Secret variables used in scripts are masked in the output, but only for exact-match values. Avoid transforming secrets (for example, base64-encoding and printing) because the transformed text will not be masked.

2.4 Artifact sources and primary artifact selection

A service can define multiple artifact sources. This matters for two advanced scenarios:

1. **Multiple registries or repositories** for the same service (for example, a mirror in each region).
2. **Environment-specific artifact selection** (for example, a hardened base image for production).

```
artifacts:
  primary:
    primaryArtifactRef: <+input>
    sources:
      - identifier: registry_us
        type: DockerRegistry
        spec:
          connectorRef: registry_us
          imagePath: acme/orders-api
          tag: <+input>
      - identifier: registry_eu
        type: DockerRegistry
        spec:
          connectorRef: registry_eu
          imagePath: acme/orders-api
          tag: <+input>
```

At run time the operator chooses `registry_us` or `registry_eu`, then chooses a tag. In automated flows, an input set or trigger supplies both.

2.4.1 Tag selection patterns

Pattern	When to use
Runtime input `<+input>`	Operator or trigger picks the tag.
`<+input>.regex(^1\.0\..*)`	Constrain allowed tags to a release line.
`<+input>.executionInput()`	Defer the choice until the stage executes.
Latest matching tag	Use a tag regex source option so the newest tag matching a pattern is selected.
Artifact trigger	New tag in the registry starts the pipeline with that tag injected.

2.4.2 Digest-based deploys

For immutability, pin deployments by image digest. When `digest` is set on the artifact source, Harness deploys `image@sha256:...` instead of a mutable tag. This protects you from a tag being overwritten between approval and rollout, which is a real risk when approvals take hours.

> **Why digests matter with long approvals**
>
> Tag `1.4.2` approved at 10:00 can point to a different image at 14:00 if someone force-pushes the tag. Digest pinning removes that class of incident. Combine it with registry-side tag immutability where available.

2.4.3 Sidecar artifacts

Sidecar artifacts let one service deploy more than one image, for example the application plus an agent container. Each sidecar has an identifier and is referenced separately, as `<+artifacts.sidecars.sidecar_id.image>`.

```
artifacts:
  primary:
```

```
    primaryArtifactRef: <+input>
    sources:
      - identifier: main_image
        type: DockerRegistry
        spec:
          connectorRef: registry_main
          imagePath: acme/orders-api
          tag: <+input>
  sidecars:
    - sidecar:
        identifier: log_shipper
        type: DockerRegistry
        spec:
          connectorRef: registry_main
          imagePath: acme/log-shipper
          tag: "2.1.0"
```

In the manifest you reference both images:

```
containers:
  - name: api
    image: <+artifacts.primary.image>
  - name: log-shipper
    image: <+artifacts.sidecars.log_shipper.image>
```

2.5 Manifest handling

2.5.1 Multiple manifests and values layering

A Kubernetes service can include several manifests, and Helm services can layer several values files. Order matters. Later files override earlier ones.

```
manifests:
  - manifest:
      identifier: chart
      type: HelmChart
      spec:
        store:
          type: Http
          spec:
            connectorRef: helm_repo
        chartName: orders-api
        chartVersion: <+input>
        helmVersion: V3
        skipResourceVersioning: false
        commandFlags:
          - commandType: Template
            flag: "--skip-tests"
  - manifest:
      identifier: values_common
      type: Values
      spec:
        store:
          type: Github
          spec:
            connectorRef: github_main
            gitFetchType: Branch
            paths:
              - deploy/values/common.yaml
            repoName: orders-api
            branch: main
```

Environment overrides can add a third values layer that wins over both (Chapter 3).

2.5.2 Git fetch types

Fetch type	Behavior	Guidance
Branch	Latest commit on branch at run time.	Fine for non-production.
Commit	Exact commit SHA.	Prefer for production reproducibility.
Tag	A Git tag.	Good for release-line manifests.

When rollback matters, a moving branch is dangerous: the rollback re-applies the previously deployed rendered state from release history, but a **re-run** of the pipeline would pick up new manifest changes. Pin by commit or tag for release pipelines.

2.5.3 Resource versioning

By default Harness appends a version suffix to ConfigMaps and Secrets it applies (for example `app-config-3`). Each deployment creates a new numbered copy. That is what lets a rolling rollback restore the exact previous configuration instead of only the previous image. You can opt out per resource with an annotation, covered in Chapter 11.

2.6 Service hooks

Service hooks run scripts at defined points during manifest handling, such as before fetching or after rendering manifests. Typical uses:

- Run a lint or policy check on rendered manifests.
- Inject a computed label.
- Decrypt a file with an external tool before apply.

Hooks are defined on the service and execute on the delegate. Keep them fast and idempotent. A failing hook fails the deployment step that triggered it.

2.7 Config files

Config files attach plain files or secret files to a service. They are most useful for non-Kubernetes deployment types (SSH, WinRM), but for Kubernetes you can reference them through expressions such as `<+configFile.getAsString("app_properties")>` to inline file contents into a ConfigMap.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: orders-app-properties
data:
  app.properties: |
    <+configFile.getAsString("app_properties")>
```

Use `getAsBase64` when embedding binary or multi-line secrets into a Kubernetes `Secret` manifest's `data` field.

2.8 Service-level checklist

Check	Why
Primary artifact tag is runtime or expression, never literal.	Avoid redeploying the same fixed build forever.
Replica counts and resource limits are service variables.	Environment overrides can tune per environment.
Secrets use the `Secret` variable type.	Masking and RBAC.
Manifests pinned by commit or tag for production.	Reproducibility.
ConfigMaps and Secrets are versioned (default).	Correct rollbacks.
Digests used where tag mutability is a risk.	Approval-to-deploy integrity.
Service names and identifiers match team conventions.	Dashboards and RBAC readability.

2.9 Common mistakes with service variables

1. **Typing numbers as strings.** If `replicas` is a `String`, arithmetic and some schema validations misbehave. Use `Number` for numeric values.
2. **Mismatched types between the service and its override.** A `Number` on the service overridden with a `String` in the environment can pass validation yet fail later in arithmetic or schema checks. Keep types identical, and declare every overridable variable on the service so its default and type are documented in one place.
3. **Referencing `service.variables` instead of `serviceVariables`.** You get the unoverridden value and your environment override seems to be ignored.
4. **Putting environment-specific literals in manifests.** Defeats reuse and forces per-environment services.
5. **Using Secret variables in places that print them.** A masked secret is safe in logs. A secret copied into a ConfigMap is not.

3. Environments, Infrastructure, and the Override Hierarchy

3.1 Environments as first-class objects

An environment captures everything about a deployment target that is not part of the application: cluster access, namespaces, variables such as region or ingress domain, and policy classification. Defining it once and reusing it across every service avoids duplicating cluster details in each pipeline.

```yaml
environment:
  name: prod_eu
  identifier: prod_eu
  description: Production, EU region
  tags:
    region: eu
    tier: prod
  type: Production
  orgIdentifier: platform
  projectIdentifier: payments
  variables:
    - name: region
      type: String
      value: eu-west-1
    - name: ingressDomain
      type: String
      value: orders.eu.example.com
    - name: minReplicas
      type: Number
      value: 4
  overrides:
    manifests: []
    configFiles: []
```

Tags on the environment matter for filtering. When you deploy to a set of environments, you can select them by tag rather than listing each one (Chapter 8).

3.1.1 Production vs PreProduction

The `type` has behavior, not only labelling:

- Dashboards and DORA-style reports separate production from non-production.
- Freeze windows can target Production environments only.
- RBAC can restrict who may deploy to Production environments.
- OPA policies can branch on `environment.type`.

Always classify honestly. Marking a staging environment as Production to "get more governance" distorts reporting.

3.2 Infrastructure definitions

An infrastructure definition describes a concrete deployment target inside an environment. For Kubernetes:

```yaml
infrastructureDefinition:
  name: prod_eu_cluster
  identifier: prod_eu_cluster
  orgIdentifier: platform
  projectIdentifier: payments
  environmentRef: prod_eu
  deploymentType: Kubernetes
  type: KubernetesDirect
  spec:
```

```
    connectorRef: k8s_prod_eu
    namespace: orders
    releaseName: release-<+INFRA_KEY_SHORT_ID>
  allowSimultaneousDeployments: false
```

3.2.1 Important infrastructure fields

Field	Meaning
`connectorRef`	Kubernetes cluster connector used by the delegate.
`namespace`	Target namespace. Can be fixed, runtime input, or an expression.
`releaseName`	Logical release identity used for release history and rollback. The default includes a short infrastructure key so two infrastructures never collide.
`allowSimultaneousDeployments`	If `false`, Harness queues concurrent deployments to the same infrastructure. See resource constraints in Chapter 6.

3.2.2 Release name and why it matters

Harness stores per-release history (what was applied, which resources, which versions) keyed by the release name in the target cluster. Rollback depends on that history. Two consequences:

- If two services share the same release name in the same namespace, they overwrite one another's history and rollback can restore the wrong state. Keep the default `release-<+INFRA_KEY_SHORT_ID>`, which is unique per service and infrastructure combination.
- If you change the release name for an existing deployment, Harness starts a new history, and a rollback has no previous successful release to return to on that first run.

3.2.3 Namespaces as expressions

You can parameterize the namespace to deploy multi-tenant or per-branch preview environments:

```
spec:
  connectorRef: k8s_dev
  namespace: orders-<+pipeline.variables.previewName>
  releaseName: release-<+INFRA_KEY_SHORT_ID>
```

Pair this with a cleanup stage or pipeline that deletes the namespace after review. Without cleanup, ephemeral namespaces accumulate.

3.3 Environment variables

Environment variables are referenced as `<+env.variables.name>`. They are the right place for facts about the target that every service needs: region, domain, feature-flag SDK key reference, logging endpoint. Each variable has a type and may be fixed, runtime, or an expression.

Use cases:

- **Hostnames** such as the ingress domain.
- **Regional parameters** such as the cloud region or the datacenter code.
- **Shared integration endpoints** such as the Datadog site or a ServiceNow instance alias.
- **Operational toggles** like `deployWindow`, referenced by conditional execution.

3.4 Overrides: the V2 model

Overrides replace values defined in the service for a specific environment, and optionally a specific service or infrastructure. Harness supports overriding:

- **Variables** (service variables).
- **Manifests** (add or replace values files and manifests).
- **Config files**.
- **Application settings and connection strings** for deployment types that use them.

The V2 override model stores overrides as their own entities, scoped by **where** they apply:

Override scope	Applies to	Typical example
Environment global	Every service deployed to the environment.	`ingressDomain` values file for `prod_eu`.
Environment and service	One service in one environment.	`orders_api` replicas in `prod_eu`.
Infrastructure and service	One service on one specific infrastructure in an environment.	A per-cluster tuning for `orders_api` on a specific cluster.
Service (defaults)	The service definition itself.	`replicas: 2`.

3.4.1 Precedence

When the same name is defined at several levels, the most specific scope wins. Lowest to highest priority:

```
  service definition                    (lowest)
       <  environment global override
       <  environment + service override
       <  infrastructure + service override   (highest)
```

> **Verify precedence in your account**
>
> The override model has been revised across Harness releases and an account can still be on the earlier "V1" environment override behavior. In V1, the environment-level service override section in the environment settings is the main mechanism. If you are not sure which model applies, create a harmless variable at two levels, deploy to a sandbox environment, and print `<+serviceVariables.name>` in a Shell Script step. A one-minute experiment is cheaper than a wrong production value.

3.4.2 Manifest override example

Add a values file that applies to every Helm-based service deployed to `prod_eu`:

```yaml
overrides:
  manifests:
    - manifest:
        identifier: prod_eu_values
        type: Values
        spec:
          store:
            type: Github
            spec:
              connectorRef: github_main
              gitFetchType: Branch
              paths:
                - environments/prod_eu/values.yaml
              repoName: platform-config
              branch: main
```

Layering order for Helm values, from lowest to highest priority: chart defaults, service values files, environment values override, service-and-environment values override. The last file in the merged list wins on conflicts.

3.4.3 Variable override example

At the environment-and-service scope, override only `replicas` and `logLevel` for `orders_api`:

```yaml
serviceOverrides:
  environmentRef: prod_eu
  serviceRef: orders_api
  spec:
    variables:
      - name: replicas
        type: Number
        value: 6
      - name: logLevel
        type: String
        value: warn
```

When the stage deploys `orders_api` to `prod_eu`, `<+serviceVariables.replicas>` resolves to `6`. The same service deployed to `dev` still resolves to `2`.

3.5 Environment groups

An environment group is a named collection of environments. It lets a pipeline target "all production regions" without enumerating each one, and it lets you add a new region without editing pipelines.

```
environmentGroup:
  name: production_regions
  identifier: production_regions
  orgIdentifier: platform
  projectIdentifier: payments
  envIdentifiers:
    - prod_eu
    - prod_us
```

In a stage, reference the group and decide whether to deploy to all members or a chosen subset:

```
environment:
  environmentGroup:
    envGroupRef: production_regions
    deployToAll: true
```

or with explicit members:

```
environmentGroup:
  envGroupRef: production_regions
  deployToAll: false
  environments:
    - environmentRef: prod_eu
      deployToAll: true
```

Chapter 8 shows how to deploy to an environment group in parallel and how to combine it with matrix looping.

3.6 Infrastructure selection at run time

Most pipelines either name the infrastructure or take it as a runtime input.

```
environment:
  environmentRef: <+input>
  deployToAll: false
  infrastructureDefinitions: <+input>
```

When a stage is used for several environments through a pipeline template, runtime inputs let one template serve dev, QA, and production. Add `allowedValues` to avoid typos:

```
environmentRef: <+input>.allowedValues(dev,qa,staging)
```

3.7 Cluster identity and safety

A frequent source of incidents is deploying to the wrong cluster because two connectors look similar. Reduce risk with these practices:

1. **Name connectors by environment and region**, such as `k8s_prod_eu`, never `k8s_cluster_2`.
2. **Restrict connector use with RBAC.** Only the release team role should reference production connectors.
3. **Use delegate selectors** that match the environment tier so a dev delegate cannot even attempt a production API call.
4. **Add an explicit guard step** in production stages: a Shell Script that asserts the kube context or cluster name equals the expected value from the environment variable.

```yaml
- step:
    type: ShellScript
    name: Assert Cluster
    identifier: assert_cluster
    timeout: 2m
    spec:
      shell: Bash
      onDelegate: true
      source:
        type: Inline
        spec:
          script: |
            expected="<+env.variables.expectedClusterName>"
            actual=$(kubectl config current-context 2>/dev/null || echo unknown)
            echo "Expected: $expected  Actual: $actual"
            test "$expected" = "$actual"
      environmentVariables: []
      outputVariables: []
```

This step is cheap insurance. If the assertion fails, the stage fails before any change reaches the cluster.

3.8 What belongs where: a decision table

Value	Put it in	Reason
Image repository path	Service artifact source	Intrinsic to the application.
Image tag	Runtime input or CI expression	Changes every build.
Default replica count	Service variable	A sane default for any environment.
Production replica count	Environment+service override	Environment-specific tuning.
Region, domain	Environment variable	Same for every service in that environment.
Cluster, namespace	Infrastructure definition	Concrete target.
Rollout approval policy	Stage, template, or OPA policy	Process, not configuration.
Secrets	Harness secret, referenced by variable	Masking and access control.

4. Variables, Expressions, Runtime Inputs, and Input Sets

4.1 Variable scopes

Harness variables exist at several scopes. Choose the narrowest scope that fits.

Scope	Declared on	Referenced as	Typical use
Account, org, project	Variable resource	`<+variable.account.name>`, `<+variable.org.name>`, `<+variable.name>`	Shared constants such as a Slack channel or a registry hostname.
Pipeline	`pipeline.variables`	`<+pipeline.variables.name>`	Run-wide parameters: ticket number, release name.
Stage	`stage.variables`	`<+stage.variables.name>`	Values specific to one stage.
Service	`serviceDefinition.spec.variables`	`<+serviceVariables.name>`	Application configuration defaults.
Environment	`environment.variables`	`<+env.variables.name>`	Target facts.
Step	Step outputs and `outputVariables`	`<+execution.steps.id.output.outputVariables.name>`	Data passed between steps.

Project and org variables (non-secret shared values) are useful for fleet-wide constants, such as the URL of an internal artifact mirror. Reference them by scope: `<+variable.account.mirrorUrl>`.

4.2 Fixed, runtime, and expression values

Every variable or field can hold one of three kinds of value.

```
variables:
  - name: tier
    type: String
    value: gold                 # fixed
  - name: releaseTicket
    type: String
    value: <+input>             # runtime input
  - name: deployNotes
    type: String
    value: "Release <+pipeline.sequenceId> to <+env.name>"   # expression
```

4.2.1 Runtime input modifiers

The `<+input>` marker accepts chained modifiers that improve safety and UX:

Modifier	Example	Effect
`.default(x)`	`<+input>.default(dev)`	Pre-fills the prompt.
`.allowedValues(a,b)`	`<+input>.allowedValues(dev,qa,prod)`	Renders a dropdown and rejects other values.
`.regex(pattern)`	`<+input>.regex(^CHG[0-9]{7}$)`	Validates the value against a regular expression.
`.executionInput()`	`<+input>.executionInput()`	Prompts at execution time instead of before the run starts.
`.default(x).allowedValues(...)`	combined	Both behaviors.

```
variables:
  - name: changeNumber
    type: String
    value: <+input>.regex(^CHG[0-9]{7}$)
  - name: targetTier
    type: String
    value: <+input>.default(qa).allowedValues(dev,qa,staging,prod)
```

> **Validate at the edge**
>
> A regex on the change number input catches a typo at the moment of starting the run, instead of failing 40 minutes later at the ServiceNow approval step.

4.2.2 Execution-time inputs

`executionInput()` pauses a stage or step at execution time and waits for a person to supply values. It is useful when the right answer depends on what happened earlier, for example "enter the canary percentage after reviewing the dashboards".

```
- step:
    type: K8sCanaryDeploy
    name: Canary Deploy
    identifier: canary_deploy
    timeout: 10m
    spec:
      instanceSelection:
        type: Count
        spec:
          count: <+input>.executionInput().default(1)
      skipDryRun: false
```

When the step is reached, the pipeline pauses with a prompt. Without a timeout and an authorized responder, a run can wait indefinitely, so pair execution inputs with a timeout, and with an `onFailure` strategy.

4.3 Expressions beyond substitution

4.3.1 Conditions and ternary expressions

Expressions can evaluate boolean logic and return one of two values:

```
<+ <+env.type> == "Production" ? "strict" : "relaxed" >
<+ <+pipeline.variables.tier> == "gold" && <+env.name> != "dev" >
<+ <+serviceVariables.replicas> > 3 >
```

Use these in `when` conditions and to pick a value inline:

```
variables:
  - name: verifyDuration
    type: String
    value: <+ <+env.type> == "Production" ? "15m" : "5m" >
```

4.3.2 String and JSON helpers

Several helper methods are available on string expressions:

Expression	Result
`<+pipeline.variables.name.toLowerCase()>`	Lower-cased value (use the standard Java-style string methods on string values).
`<+pipeline.variables.name.trim()>`	Trimmed.
`<+pipeline.variables.name.replace("a","b")>`	Replacement.
`<+json.format(<+some.object>)>`	Serialize an object.

Expression	Result
`<+json.select("$.items[0].id", <+jsonString>)>`	JSONPath selection from a JSON string.
`<+json.object(<+jsonString>)>`	Parse a JSON string into an object for traversal.

JSON helpers pair well with the Http step, which can return a response body:

```
- step:
    type: Http
    name: Get Release Metadata
    identifier: get_release_metadata
    timeout: 30s
    spec:
      url: https://releases.example.com/api/v1/releases/<+pipeline.variables.releaseId>
      method: GET
      headers: []
      outputVariables:
        - name: owner
          type: String
          value: <+json.select("$.owner", <+httpResponseBody>)>
      assertion: <+httpResponseCode> == 200
```

Later steps read it as `<+execution.steps.get_release_metadata.output.outputVariables.owner>`.

4.3.3 Output variables

Shell Script steps declare output variables from shell variables:

```
- step:
    type: ShellScript
    name: Compute Canary Size
    identifier: compute_canary_size
    timeout: 5m
    spec:
      shell: Bash
      onDelegate: true
      source:
        type: Inline
        spec:
          script: |
            replicas=<+serviceVariables.replicas>
            canary=$(( (replicas * 10 + 99) / 100 ))
            [ "$canary" -lt 1 ] && canary=1
            export canaryCount=$canary
      outputVariables:
        - name: canaryCount
          type: String
          value: canaryCount
      environmentVariables: []
```

Reference it in a following step with `<+execution.steps.compute_canary_size.output.outputVariables.canaryCount>`. Because output variables from Shell Script steps are strings, convert where numbers are required. In a K8s canary step you can often supply the expression directly where a number is expected.

> **Scope of step identifiers**
>
> Expressions that point at a step by identifier are sensitive to renames and to step groups and looping. In a looped stage, identifiers carry a suffix per iteration. Use `<+strategy.identifierPostFix>` when you must address a specific iteration (Chapter 7), and avoid renaming identifiers in a pipeline that other pipelines or templates reference.

4.3.4 Referencing earlier stages

To read a value produced by an earlier stage:

```
<+pipeline.stages.deploy_staging.spec.execution.steps.compute_canary_size.output.outputVariables.canaryCount>
```

This long path is the stable form. In a template-based pipeline, prefer to capture the value into a pipeline variable via an output variable at stage level and reference that, which hides the nesting.

4.4 Secrets in expressions

Form	Meaning
`<+secrets.getValue("name")>`	Secret at project scope.
`<+secrets.getValue("org.name")>`	Secret at org scope.
`<+secrets.getValue("account.name")>`	Secret at account scope.
`<+secrets.getValue(<+serviceVariables.dbPassword>)>`	Secret named by a variable.

Secrets are masked in logs, but masking is string-based. If a script prints a modified form (for example, a substring or an encoded form), the mask does not apply. Treat output carefully.

4.5 Input sets

An input set is a saved collection of runtime input values for a specific pipeline. It allows consistent re-runs and unattended triggers.

```yaml
inputSet:
  name: prod_eu_release
  identifier: prod_eu_release
  orgIdentifier: platform
  projectIdentifier: payments
  pipeline:
    identifier: orders_release
    variables:
      - name: releaseTicket
        type: String
        value: CHG0034567
    stages:
      - stage:
          identifier: deploy_prod
          type: Deployment
          spec:
            service:
              serviceRef: orders_api
              serviceInputs:
                serviceDefinition:
                  type: Kubernetes
                  spec:
                    artifacts:
                      primary:
                        primaryArtifactRef: main_image
                        sources:
                          - identifier: main_image
                            type: DockerRegistry
                            spec:
                              tag: 1.4.2
```

4.5.1 Overlay input sets

An **overlay input set** composes several input sets in order. Later sets override earlier ones. This allows reusable building blocks:

Input set	Contains
`base_orders`	Pipeline variables common to all runs.
`env_prod_eu`	Environment and infrastructure selections.
`release_1_4_2`	The artifact tag.

An overlay `prod_eu_1_4_2` lists `base_orders`, `env_prod_eu`, `release_1_4_2`. Changing the tag means editing or swapping a single small input set.

```
overlayInputSet:
  name: prod_eu_1_4_2
  identifier: prod_eu_1_4_2
  orgIdentifier: platform
  projectIdentifier: payments
  pipeline:
    identifier: orders_release
  inputSetReferences:
    - base_orders
    - env_prod_eu
    - release_1_4_2
```

4.5.2 Input sets and triggers

Triggers reference input sets by identifier, or define inline input values. A webhook trigger for tag pushes commonly fills the artifact tag from the payload with an expression, for example `<+trigger.payload.ref>`, while environment inputs come from a stable input set.

4.5.3 Input set maintenance

Input sets are tied to the pipeline's input schema. Adding a runtime input to the pipeline makes existing input sets incomplete. Harness flags invalid input sets in the UI. Practice:

- Prefer **few, composable** input sets over many large ones.
- Keep a naming convention, for example `<env>_<purpose>`.
- Store pipelines and input sets in Git (remote entities) so reviews cover both.

4.6 Pipeline variables vs input sets vs service variables

Question	Use
Value belongs to the application and has a sane default.	Service variable.
Value differs per environment.	Environment variable or override.
Value is chosen per run.	Pipeline variable as runtime input.
The same combination of run-time choices repeats.	Input set.
Value is derived from earlier work.	Output variable plus expression.
Value must be chosen mid-run by a human.	`executionInput()`.

4.7 Debugging expressions

When an expression resolves unexpectedly, check in this order:

1. **Spelling and namespace.** `serviceVariables` vs `service.variables`; `env` vs `environment`.
2. **Timing.** Is the referenced output produced before the step that reads it?
3. **Looping suffixes.** In a matrix, identifiers may have suffixes.
4. **Scope of the identifier.** Steps in a step group still resolve by identifier but may need the group path in templates.
5. **Print it.** A Shell Script step with `echo "value=<+expr>"` is the fastest probe.
6. **Execution view.** The step's *Input* tab shows the resolved values Harness actually used.

```
- step:
    type: ShellScript
    name: Probe
    identifier: probe
    timeout: 1m
    spec:
      shell: Bash
      onDelegate: true
```

```
      source:
        type: Inline
        spec:
          script: |
            echo "service=<+service.name>"
            echo "env=<+env.name> type=<+env.type>"
            echo "ns=<+infra.namespace> release=<+infra.releaseName>"
            echo "tag=<+artifact.tag> image=<+artifact.image>"
            echo "replicas=<+serviceVariables.replicas>"
      environmentVariables: []
      outputVariables: []
```

Keep a Probe step in a sandbox pipeline. It pays for itself the first time an override does not behave as expected.

5. Kubernetes Deployment Strategies and Step Reference

5.1 How Harness deploys to Kubernetes

For Kubernetes deployment types, the delegate performs these phases for each deployment step:

1. **Fetch** manifests and values from their stores, and resolve the artifact.
2. **Render** manifests (plain, Helm template, Kustomize build, or OpenShift process), substituting expressions.
3. **Dry run** (optional): `kubectl apply --dry-run` against the cluster to validate the manifests.
4. **Apply** manifests to the namespace, tracking the resources in a release record.
5. **Steady state check**: wait until the managed workloads reach a healthy state, or time out.
6. **Record** the release: the resources, their versions, and the status of the release.

The release record created in phase 6 is the foundation for rollback (Chapter 11).

5.1.1 Managed workloads

Harness identifies a **managed workload** among the rendered resources. Rolling and canary strategies track it for steady state and for scaling. By default a workload kind such as Deployment, StatefulSet, DaemonSet, or Job qualifies. If your manifest set has more than one Deployment, annotate the one that should be managed, as shown below. Without this, Harness reports an error or manages an unintended resource.

```
metadata:
  annotations:
    harness.io/managed-workload: "true"
```

If you want Harness to apply a resource but not wait on it or version it, use the resource annotations:

Annotation	Effect
`harness.io/managed-workload: "true"`	Mark this workload as the managed one for the step.
`harness.io/skip-versioning: "true"`	Do not version this ConfigMap or Secret.
`harness.io/direct-apply: "true"`	Apply directly and do not treat as the managed workload.
`harness.io/primary-service: "true"`	Blue-green: the service that receives live traffic.
`harness.io/stage-service: "true"`	Blue-green: the service pointing at the new version before swap.
`harness.io/skip-pruning: "true"`	Do not delete this resource when pruning is enabled.

> **Annotation names**
> The annotations above are the commonly used set. Harness has adjusted annotation handling across releases, so confirm the exact names for your delegate version in the Harness Kubernetes deployment documentation when adopting any of them in production manifests.

5.2 Rolling deployment

Rolling is the default and simplest strategy: Kubernetes replaces Pods incrementally according to the Deployment's `strategy.rollingUpdate` settings, and Harness waits for steady state.

```
execution:
  steps:
    - step:
        name: Rollout Deployment
        identifier: rolloutDeployment
        type: K8sRollingDeploy
```

```
        timeout: 10m
        spec:
          skipDryRun: false
          pruningEnabled: false
    rollbackSteps:
      - step:
          name: Rollback Rollout Deployment
          identifier: rollbackRolloutDeployment
          type: K8sRollingRollback
          timeout: 10m
          spec:
            pruningEnabled: false
```

5.2.1 Key options

Option	Effect
`skipDryRun`	Skip the dry run validation. Leave `false` in production.
`pruningEnabled`	Delete resources that were in the previous release but are no longer in the manifests. Powerful, handle with care.
`skipSteadyStateCheck`	Do not wait for steady state. Rarely right; hides failures.
`timeout`	Maximum duration for the step, including the steady state wait.

5.2.2 Kubernetes-level controls

Rolling behavior is also governed by your manifest:

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 0
  minReadySeconds: 10
  progressDeadlineSeconds: 300
```

`maxUnavailable: 0` with a correct readiness probe gives zero-downtime rollouts. `progressDeadlineSeconds` makes Kubernetes mark a stuck rollout as failed, which helps Harness detect it earlier than the step timeout.

5.3 Canary deployment

A canary deployment runs a small number of new-version Pods alongside the stable version, lets you verify, then promotes. Harness canary on Kubernetes uses these steps:

1. `K8sCanaryDeploy` creates the canary workload using a **copy of the managed workload** with a suffix, sized by `instanceSelection`.
2. Verification and approval steps (your choice).
3. `K8sCanaryDelete` removes the canary workload.
4. `K8sRollingDeploy` performs the full rollout of the new version.

```
execution:
  steps:
    - stepGroup:
        name: Canary Deployment
        identifier: canaryDepoyment
        steps:
          - step:
              name: Canary Deployment
              identifier: canaryDeployment
              type: K8sCanaryDeploy
              timeout: 10m
              spec:
                instanceSelection:
                  type: Count
                  spec:
```

```
                count: 1
              skipDryRun: false
        - step:
            name: Canary Delete
            identifier: canaryDelete
            type: K8sCanaryDelete
            timeout: 10m
            spec: {}
    - stepGroup:
        name: Primary Deployment
        identifier: primaryDepoyment
        steps:
          - step:
              name: Rolling Deployment
              identifier: rollingDeployment
              type: K8sRollingDeploy
              timeout: 10m
              spec:
                skipDryRun: false
  rollbackSteps:
    - step:
        name: Canary Delete
        identifier: rollbackCanaryDelete
        type: K8sCanaryDelete
        timeout: 10m
        spec: {}
    - step:
        name: Rolling Rollback
        identifier: rollingRollback
        type: K8sRollingRollback
        timeout: 10m
        spec: {}
```

5.3.1 Instance selection

Type	Example	Meaning
`Count`	`count: 2`	Exactly two canary Pods.
`Percentage`	`percentage: 20`	Percent of the desired replica count, rounded up.

When the replica count comes from a service variable, a percentage keeps the canary proportional across environments: 10 percent of 2 replicas is 1 Pod, while 10 percent of 50 is 5.

5.3.2 Where verification goes

Insert a **Verify** step (Chapter 10) between `K8sCanaryDeploy` and `K8sCanaryDelete`. If verification fails, the failure strategy triggers rollback, which runs `K8sCanaryDelete` and leaves the stable version untouched. The user-facing impact of a bad canary is limited to the traffic share the canary Pods receive.

```
K8sCanaryDeploy  ->  Verify (Datadog)  ->  Approval (optional)  ->  K8sCanaryDelete
                                                                          |
                                                                          v
                                                                   K8sRollingDeploy
```

5.3.3 Canary traffic share

With a plain Kubernetes Service selecting both stable and canary Pods by label, traffic split is proportional to Pod count. One canary Pod out of ten gives roughly ten percent. For precise percentages independent of Pod count, use a service mesh or ingress controller with weighted routing and drive the weights from Harness steps (Shell Script or Http steps) around the canary.

5.4 Blue-green deployment

Blue-green runs two complete versions and switches traffic at once. In Harness Kubernetes blue-green:

- `K8sBlueGreenDeploy` creates the new (stage) workload and a stage Service.
- A verification window follows, where you can test the stage Service.
- `K8sBGSwapServices` swaps selectors so the primary Service points at the new workload.

```yaml
execution:
  steps:
    - step:
        name: Stage Deployment
        identifier: stageDeployment
        type: K8sBlueGreenDeploy
        timeout: 10m
        spec:
          skipDryRun: false
          pruningEnabled: false
    - step:
        name: Approval Before Swap
        identifier: approvalBeforeSwap
        type: HarnessApproval
        timeout: 1h
        spec:
          approvalMessage: Validate the stage service, then approve the swap.
          includePipelineExecutionHistory: true
          approvers:
            userGroups:
              - account._account_all_users
            minimumCount: 1
            disallowPipelineExecutor: false
    - step:
        name: Bluegreen Swap Services
        identifier: bluegreenSwapServices
        type: K8sBGSwapServices
        timeout: 10m
        spec: {}
  rollbackSteps:
    - step:
        name: Swap Primary with Stage service
        identifier: swapPrimaryWithStageService
        type: K8sBGSwapServices
        timeout: 10m
        spec: {}
```

5.4.1 Two services and the swap

Your manifests need two Services (or Harness generates one stage Service). The primary Service carries live traffic. After `K8sBlueGreenDeploy`, the stage Service points at the new workload, while the primary still points at the old one. The swap exchanges the selectors. After a swap, the **old** workload remains running behind the stage Service, which means rollback is another swap, which is nearly instant.

5.4.2 Cost and scale-down

Blue-green doubles capacity during the window. Harness scales down the previous version after a successful swap, according to its scale-down behavior for the old workload. Plan cluster capacity for the overlap, especially for stateful dependencies and database connection limits.

5.5 Other Kubernetes steps you will use

Step type	Purpose
`K8sApply`	Apply manifests as-is, with optional wait for steady state.
`K8sDelete`	Delete resources by manifest path, resource name, or release.
`K8sScale`	Scale a workload to a count or percentage, optionally scale down after.
`K8sDryRun`	Validate manifests without applying.

Step type	Purpose
`K8sRollingRollback`	Revert a rolling deployment to the previous release.
`K8sCanaryDelete`	Remove canary resources.
`K8sBGSwapServices`	Swap blue-green selectors.
`K8sBlueGreenStageScaleDown`	Scale down the stage workload after swap.
`HelmDeploy`, `HelmRollback`	For native Helm (as opposed to Helm charts rendered through the Kubernetes steps).
`ShellScript`, `Http`	Orchestration and integration glue.

5.5.1 Kubernetes versus Native Helm

Harness has two ways to deploy Helm charts. They look similar and behave very differently.

Aspect	Kubernetes deployment type with Helm chart manifest	Native Helm deployment type
Who applies resources	Harness (renders chart, applies with kubectl-style logic).	Helm itself (install or upgrade).
Release tracking	Harness release record, with versioned ConfigMaps and Secrets.	Helm release history in the cluster.
Rollback	`K8sRollingRollback` and friends.	`HelmRollback` using `helm rollback`.
Canary and blue-green	Supported through the Harness K8s steps.	Supported through Harness-specific Helm steps.

Choose per application and stay consistent, because the rollback semantics differ. Mixing the two for the same release name in the same namespace is a recipe for confusion.

5.6 Choosing a strategy

Requirement	Prefer
Simple, stateless service, strong readiness probes	Rolling
Need metric-based confidence before full rollout	Canary with Verify
Need instant cutover and instant revert	Blue-green
Database schema changes that are not backward compatible	Blue-green with an expand and contract migration plan, or maintenance window
Cost-sensitive, many services	Rolling plus verification
Strict compliance with human sign-off before traffic moves	Blue-green with approval before swap

> **Schema compatibility is outside any strategy's control**
>
> No deployment strategy protects you from an incompatible database migration. Use the expand and contract pattern: add new columns or tables first (compatible with the old code), deploy the new code, and remove old structures in a later release. Rollbacks only work when the old code can run against the current schema.

5.7 Timeouts and steady state

Timeouts in Harness cascade: a step timeout bounds that step, a stage has no inherent timeout unless you set a failure strategy or the pipeline has one, and approvals have their own timeout.

Recommendations:

- Set `timeout` explicitly on every deploy step. Defaults are generous and can hide stuck rollouts.
- Set `progressDeadlineSeconds` in the manifest to a value shorter than the step timeout.
- Include image pull time in your estimate for large images and cold nodes.
- Use `K8sApply` with `skipSteadyStateCheck: false` for custom resources that expose readiness, and consider an explicit wait step for resources whose readiness Harness cannot infer.

6. Barriers and Resource Constraints

Flow control in Harness covers two different problems that sound alike:

- **Barriers** synchronize parallel branches within one pipeline run so that one branch does not proceed until another reaches a defined point.
- **Resource constraints** serialize or limit concurrent runs that would otherwise collide on a shared resource, such as an infrastructure definition.

6.1 What a barrier is

A barrier is a named synchronization point declared once at the pipeline level and referenced by **Barrier steps** placed in parallel stages or parallel steps. Execution of each branch pauses at its Barrier step until **all** Barrier steps that reference the same barrier have been reached. Then all of them proceed.

```
   Stage A (backend)                    Stage B (database migration)
   ----------------                     ----------------------------
   deploy backend v2                    run migration
        |                                    |
   [Barrier: db_ready] <------ waits ------> [Barrier: db_ready]
        |                                    |
   smoke tests                          post-migration checks
```

Both branches reach `db_ready`. Neither proceeds until both have arrived.

6.1.1 When to use a barrier

Scenario	Why a barrier helps
Deploying an API and a worker that must switch behavior at the same moment.	Both reach the barrier after their pre-checks, then both continue to the cutover step together.
Database migration and application deployment in parallel stages.	App rollout waits until migration finishes.
Multi-region production release	Do not start region two's cutover until region one's pre-checks completed and vice versa, so a failure in one region prevents a half-done fleet.
Parallel verification and warm-up	Warm caches in one stage and finish verification in another before traffic opens.

6.1.2 When a barrier is the wrong tool

- **Ordering of sequential stages.** Stages already run in order. A barrier adds nothing.
- **Passing data between branches.** Barriers synchronize, they do not carry values. Use outputs after the barrier, or pipeline-level variables.
- **Preventing two pipeline runs from deploying at the same time.** Use resource constraints or concurrency limits (section 6.6).

6.2 Defining and using barriers

6.2.1 Declare at pipeline level

```yaml
pipeline:
  name: orders-prod-release
  identifier: orders_prod_release
  projectIdentifier: payments
  orgIdentifier: platform
  flowControl:
    barriers:
      - name: db_ready
        identifier: db_ready
```

```
      - name: cutover_gate
        identifier: cutover_gate
```

Each barrier has a name and an identifier. The identifier is what steps reference.

6.2.2 Reference in a Barrier step

```
- step:
    type: Barrier
    name: Wait for DB
    identifier: wait_for_db
    timeout: 30m
    spec:
      barrierRef: db_ready
```

The `timeout` governs how long this step waits for the other participants. If the timeout expires, the step fails with a barrier timeout and your failure strategy takes over.

6.2.3 Full example: migration and app in parallel stages

```
pipeline:
  name: orders-release-with-barrier
  identifier: orders_release_with_barrier
  projectIdentifier: payments
  orgIdentifier: platform
  flowControl:
    barriers:
      - name: db_ready
        identifier: db_ready
  stages:
    - parallel:
        - stage:
            name: migrate_db
            identifier: migrate_db
            type: Deployment
            spec:
              deploymentType: Kubernetes
              service:
                serviceRef: orders_migrations
              environment:
                environmentRef: staging
                deployToAll: false
                infrastructureDefinitions:
                  - identifier: staging_cluster
              execution:
                steps:
                  - step:
                      type: K8sApply
                      name: Run Migration Job
                      identifier: run_migration_job
                      timeout: 20m
                      spec:
                        filePaths:
                          - migrations/job.yaml
                        skipDryRun: false
                        skipSteadyStateCheck: false
                        skipRendering: false
                        overrides: []
                  - step:
                      type: Barrier
                      name: Migration Done
                      identifier: migration_done
                      timeout: 30m
                      spec:
                        barrierRef: db_ready
                rollbackSteps: []
        - stage:
```

```
          name: deploy_app
          identifier: deploy_app
          type: Deployment
          spec:
            deploymentType: Kubernetes
            service:
              serviceRef: orders_api
            environment:
              environmentRef: staging
              deployToAll: false
              infrastructureDefinitions:
                - identifier: staging_cluster
            execution:
              steps:
                - step:
                    type: ShellScript
                    name: Pre Checks
                    identifier: pre_checks
                    timeout: 5m
                    spec:
                      shell: Bash
                      onDelegate: true
                      source:
                        type: Inline
                        spec:
                          script: echo "pre-checks ok"
                      environmentVariables: []
                      outputVariables: []
                - step:
                    type: Barrier
                    name: Wait For Migration
                    identifier: wait_for_migration
                    timeout: 30m
                    spec:
                      barrierRef: db_ready
                - step:
                    type: K8sRollingDeploy
                    name: Rollout
                    identifier: rollout
                    timeout: 10m
                    spec:
                      skipDryRun: false
                      pruningEnabled: false
              rollbackSteps:
                - step:
                    type: K8sRollingRollback
                    name: Rollback
                    identifier: rollback
                    timeout: 10m
                    spec:
                      pruningEnabled: false
```

Notice that `deploy_app` runs its pre-checks while the migration is still running. The Barrier step is placed right before the action that must wait. That saves wall-clock time versus running everything sequentially.

6.3 Barrier rules and constraints

Rule	Detail
Barriers are pipeline-scoped.	Declared under `flowControl.barriers`. A barrier cannot span pipelines.
Participants must be in parallel branches.	A barrier only makes sense when branches run concurrently. Placing two Barrier steps for the same barrier in sequential stages produces a deadlock: the first waits forever for the second which never starts.
Every declared barrier needs at least two references to be meaningful.	A barrier referenced by only one step does nothing useful.
Participants arrive and wait.	Steps that reach the barrier first wait until the others arrive or the timeout fires.

Rule	Detail
Timeouts matter.	If one branch fails before reaching the barrier, others wait until their own barrier step timeout.

The classic barrier deadlock

Branch B fails before it reaches the barrier. Branch A sits at the barrier until its timeout (30 minutes in the example). The pipeline looks hung. Mitigations: keep barrier timeouts short and realistic, and ensure the failure strategy for the failing branch aborts or fails the whole pipeline so waiting branches are released. Harness fails waiting Barrier steps when the other participant fails, but long timeouts still delay visibility of the real cause. Always read the failed branch first.

6.3.1 Barrier and failure interaction

If a participant in a barrier group fails before reaching the barrier, the waiting participants are failed (the barrier cannot be satisfied). Rollback steps for the waiting stage then run according to its failure strategy. Design rollbacks as if any branch can stop at any point.

6.3.2 Barriers with step-level parallelism

You can also place barriers in parallel **steps** inside a single stage:

```
steps:
  - parallel:
      - step:
          type: ShellScript
          name: Warm Cache
          identifier: warm_cache
          timeout: 10m
          spec:
            shell: Bash
            onDelegate: true
            source:
              type: Inline
              spec:
                script: ./warm-cache.sh
            environmentVariables: []
            outputVariables: []
      - step:
          type: ShellScript
          name: Provision Queue
          identifier: provision_queue
          timeout: 10m
          spec:
            shell: Bash
            onDelegate: true
            source:
              type: Inline
              spec:
                script: ./provision-queue.sh
            environmentVariables: []
            outputVariables: []
```

Barrier steps inside parallel step branches synchronize them the same way as across stages. Use stage-level barriers when stages have different services or environments, and step-level when the work shares one service context.

6.4 Barriers across multiple services and regions

Barriers become powerful when combined with the multi-service and matrix features in Chapters 7 and 8. A frequent need is "do not cut over any region until every region finished its pre-checks".

6.4.1 Pattern: all regions ready, then cutover

```
prod_eu: deploy canary -> verify -> [BARRIER all_regions_ready] -> full rollout
prod_us: deploy canary -> verify -> [BARRIER all_regions_ready] -> full rollout
```

With a parallel pair of stages (one per region), each includes a Barrier step referencing `all_regions_ready`. If both canaries verify, both proceed to full rollout together. If either fails verification, the other is released as failed and does not proceed.

6.4.2 Barriers inside looped stages

When a stage uses a matrix or repeat strategy, the stage is expanded into several concurrent copies. A Barrier step inside that stage is expanded too, and every copy references the same barrier identifier. Harness treats all copies as participants. This gives you "wait for all iterations" semantics without enumerating regions by hand.

> **Participant counting in looped stages**
>
> A barrier waits for all expected participants. If you set `maxConcurrency` lower than the number of iterations, iterations beyond the limit have not started yet and cannot reach the barrier, which can deadlock the iterations that did start. When using barriers inside looped stages, keep `maxConcurrency` unset or at least equal to the number of iterations. Confirm the behavior in a sandbox pipeline before you rely on it in production.

6.5 Designing with barriers: a checklist

1. Draw the parallel branches and mark the exact action that must wait.
2. Place the Barrier step immediately before that action, not at the start of the branch.
3. Choose timeouts based on the slowest realistic branch plus margin.
4. Set failure strategies so a failure in one branch releases the others quickly.
5. Make rollback steps independent. Each branch should roll back its own work.
6. Test the failure path: force one branch to fail and watch the others.

6.6 Resource constraints

A barrier coordinates within a run. **Resource constraints** coordinate across runs.

6.6.1 The problem

Two engineers start a deployment of the same service to the same infrastructure at the same time. Both fetch manifests, both apply, both update the release record. At best the later one wins. At worst the release history is corrupted and a rollback restores the wrong version.

6.6.2 Infrastructure-level protection

On an infrastructure definition, `allowSimultaneousDeployments: false` (the default) makes Harness queue deployments targeting the same infrastructure key (service plus infrastructure). The second run waits until the first finishes. The wait shows up in the execution as a queued state.

```
infrastructureDefinition:
  identifier: prod_eu_cluster
  environmentRef: prod_eu
  deploymentType: Kubernetes
  type: KubernetesDirect
  spec:
    connectorRef: k8s_prod_eu
    namespace: orders
    releaseName: release-<+INFRA_KEY_SHORT_ID>
  allowSimultaneousDeployments: false
```

Set `allowSimultaneousDeployments: true` only for ephemeral or preview environments where overlapping runs are acceptable and release history is not relied on.

6.6.3 The Queue step

A **Queue** step lets you define your own resource key and serialize any section of a pipeline. Everything that uses the same key is serialized.

```
- step:
    type: Queue
```

```
  name: Serialize Prod Release
  identifier: serialize_prod_release
  timeout: 1h
  spec:
    key: prod-release-<+env.identifier>
    scope: Pipeline
```

Field	Meaning
`key`	The resource key. Any pipeline using the same key participates.
`scope`	How widely the key is shared. `Pipeline` limits it to this pipeline. Wider scopes (such as `Project`) let different pipelines share the same key. Available scope values depend on your release.

Use a Queue step when two **different pipelines** must not run a critical section simultaneously, for example a release pipeline and a database maintenance pipeline that both touch production.

> **Queue vs infrastructure queueing**
>
> Infrastructure queueing protects one service on one infrastructure. A Queue step protects any logical resource you name, such as `payments-db-maintenance`. Use infrastructure queueing as the default and add Queue steps for cross-pipeline critical sections.

6.6.4 Concurrency and pipeline-level limits

Pipelines can also be configured so that a new execution waits or the older one is aborted when a second execution of the same pipeline starts. Use this for pipelines where only the latest commit matters, such as a preview environment deployment from a pull request. For release pipelines, queueing is almost always safer than aborting.

6.7 Barrier vs resource constraint vs approval

Need	Tool
Two parallel branches must meet before continuing.	Barrier
Only one run at a time may touch a resource.	Infrastructure queue or Queue step
A human must decide whether to continue.	Approval
A metric-based decision whether to continue.	Verify step
Do not run in a given time period.	Freeze window or conditional execution on time

6.8 Troubleshooting barriers

Symptom	Likely cause	Fix
Pipeline stuck at a Barrier step.	Another participant has not arrived, or only one step references the barrier.	Check that at least two Barrier steps reference the same `barrierRef` and that all participants run in parallel.
Barrier times out although both branches ran.	Participants were not concurrent: one branch finished before the other started.	Put both branches in the same `parallel` block.
"Barrier not found" validation error.	The barrier is not declared in `flowControl.barriers`, or the identifier differs.	Declare it and match identifiers exactly.
All branches fail after one branch fails early.	Expected: waiting participants are released as failed when a peer cannot arrive.	Fix the original failing branch.
Barrier inside matrix deadlocks.	`maxConcurrency` lower than iteration count.	Remove or raise the limit.

7. Looping Strategies: Matrix, Repeat, and Parallelism

7.1 Why loop

A pipeline is a template for work. Looping strategies let one stage or step definition expand into many concurrent or sequential executions at run time, each with its own values. Without looping, "deploy five services to four regions" is twenty copy-pasted stages. With looping, it is one stage and a matrix.

Harness provides three strategies under the `strategy` key:

Strategy	Expands by	Typical use
`matrix`	Cartesian product of named axes	Region by environment, service by version, test shard by platform.
`repeat`	Items in a list, or a fixed count	Per-tenant steps, canary phases, retries by item.
`parallelism`	N identical copies	Test sharding, load generation.

The `strategy` block can be placed on a **stage**, a **step**, or a **step group**.

7.2 Matrix

7.2.1 Basic syntax

```
strategy:
  matrix:
    region: [eu, us]
    tier: [web, api]
    maxConcurrency: 2
```

This produces four combinations: `eu/web`, `eu/api`, `us/web`, `us/api`. `maxConcurrency` limits how many run at once. Omit it for unbounded parallelism (Harness applies its own platform limits).

7.2.2 Referencing matrix values

Inside the looped stage or step, each axis is available as `<+matrix.axisName>`:

```
- stage:
    name: deploy_region
    identifier: deploy_region
    type: Deployment
    strategy:
      matrix:
        region: [eu, us]
        maxConcurrency: 2
    spec:
      deploymentType: Kubernetes
      service:
        serviceRef: orders_api
      environment:
        environmentRef: prod_<+matrix.region>
        deployToAll: false
        infrastructureDefinitions:
          - identifier: prod_<+matrix.region>_cluster
      execution:
        steps:
          - step:
              type: K8sRollingDeploy
              name: Rollout <+matrix.region>
              identifier: rollout
              timeout: 10m
              spec:
                skipDryRun: false
```

```
            pruningEnabled: false
      rollbackSteps:
        - step:
            type: K8sRollingRollback
            name: Rollback
            identifier: rollback
            timeout: 10m
            spec:
              pruningEnabled: false
```

A single stage definition becomes two executions: one to `prod_eu`, one to `prod_us`. Each iteration shows separately in the execution graph and has its own logs and status.

7.2.3 Excluding combinations

Not every combination of axes is valid. Use `exclude` to remove specific combinations:

```
strategy:
  matrix:
    region: [eu, us, apac]
    tier: [web, api, batch]
    exclude:
      - region: apac
        tier: batch
      - region: eu
        tier: batch
    maxConcurrency: 4
```

Here `batch` runs only in `us`. Excluded combinations are not created.

7.2.4 Matrix with runtime inputs and expressions

Matrix axes may be runtime inputs or expressions, which allows the operator to choose the targets:

```
strategy:
  matrix:
    region: <+input>.allowedValues(eu,us,apac)
```

or derived from earlier data. An axis can also come from an expression that returns a list, for example parsed JSON from an Http step:

```
strategy:
  matrix:
    tenant: <+json.list("tenants", <+pipeline.variables.tenantsJson>)>
```

> **List-producing expressions**
>
> The exact expression helpers available for turning a string into a list depend on your Harness version. If you pass a JSON array through a pipeline variable, use the supported JSON list helper for your release, or supply a literal list through an input set. Validate by running a Probe stage that prints the expanded values.

7.2.5 Naming iterations

By default each iteration is labelled by its matrix values. You can control the node name for readability:

```
strategy:
  matrix:
    region: [eu, us]
  nodeName: deploy-<+matrix.region>
```

Node names appear in the execution graph and in notifications. A good name makes it obvious which region failed.

7.3 Repeat

7.3.1 Repeat over a list

```
strategy:
  repeat:
    items: ["tenant-a", "tenant-b", "tenant-c"]
    maxConcurrency: 1
```

The items are referenced as `<+repeat.item>`, and the position as `<+repeat.index>`. With `maxConcurrency: 1`, the items run strictly one after another, which is useful when each iteration depends on the previous being stable (for example, a progressive rollout across tenants).

```
- step:
    type: ShellScript
    name: Migrate <+repeat.item>
    identifier: migrate_tenant
    timeout: 10m
    strategy:
      repeat:
        items: ["tenant-a", "tenant-b", "tenant-c"]
        maxConcurrency: 1
    spec:
      shell: Bash
      onDelegate: true
      source:
        type: Inline
        spec:
          script: |
            echo "Migrating <+repeat.item> (index <+repeat.index>)"
      environmentVariables: []
      outputVariables: []
```

7.3.2 Repeat with partitions

For large item lists, you can process in batches. Repeat supports a partition configuration so that items are grouped, with each iteration receiving a group.

```
strategy:
  repeat:
    items: <+pipeline.variables.tenants>
    maxConcurrency: 2
    partitionSize: 5
```

Each iteration then handles a slice of up to five items, available through `<+repeat.partition>`. Use this when the per-item overhead is high and the same action can run for a batch.

> **Partition syntax**
>
> Partition-related fields and the accessor name vary across releases. Confirm in the YAML editor's schema hints and the documentation for your account before standardizing on them.

7.3.3 Repeat as a rate limiter for progressive rollout

Combine `repeat` with `maxConcurrency: 1` and a Verify step to build waves:

```
wave 1: tenant-a  -> verify
wave 2: tenant-b  -> verify
wave 3: tenant-c  -> verify
```

If wave 2 fails verification, wave 3 never starts. This is a controlled blast radius.

7.4 Parallelism

`parallelism` runs N identical copies of a step or stage. It is mostly for work that partitions itself using the iteration number.

```
- step:
    type: ShellScript
    name: Load Shard
    identifier: load_shard
    timeout: 15m
    strategy:
      parallelism: 4
    spec:
      shell: Bash
      onDelegate: true
      source:
        type: Inline
        spec:
          script: |
            echo "Shard <+strategy.iteration> of <+strategy.iterations>"
            ./run-load.sh --shard <+strategy.iteration> --total <+strategy.iterations>
      environmentVariables: []
      outputVariables: []
```

`<+strategy.iteration>` is zero-based. `<+strategy.iterations>` is the total.

7.5 Strategy expression reference

Expression	Meaning
`<+matrix.axis>`	Value of a matrix axis for this iteration.
`<+repeat.item>`	Current item in a repeat.
`<+repeat.index>`	Zero-based index in a repeat.
`<+strategy.iteration>`	Zero-based iteration number.
`<+strategy.iterations>`	Total number of iterations.
`<+strategy.identifierPostFix>`	Suffix Harness appended to the node identifier in this iteration.
`<+strategy.node.<id>.identifierPostFix>`	Suffix of another node in the same looping group.

7.5.1 Identifiers inside iterations

Harness gives each iteration a unique runtime identifier by appending a suffix. If a downstream step must read an output of the **matching iteration** of an earlier step, build the identifier with the suffix:

```
<+execution.steps.compute_size_<+strategy.identifierPostFix>.output.outputVariables.size>
```

Without the suffix, the reference points at the base identifier, which in a looped context may not resolve.

7.6 Looping on steps and step groups

Looping on a step runs that step once per iteration, in parallel by default. Looping on a step group runs the whole group per iteration, which is how you give each iteration a deploy, verify, and approve mini-pipeline.

```
- stepGroup:
    name: Per Tenant
    identifier: per_tenant
    strategy:
      repeat:
        items: ["a", "b", "c"]
        maxConcurrency: 1
    steps:
      - step:
```

```
        type: ShellScript
        name: Deploy <+repeat.item>
        identifier: deploy_tenant
        timeout: 10m
        spec:
          shell: Bash
          onDelegate: true
          source:
            type: Inline
            spec:
              script: ./deploy-tenant.sh <+repeat.item>
          environmentVariables: []
          outputVariables: []
    - step:
        type: ShellScript
        name: Check <+repeat.item>
        identifier: check_tenant
        timeout: 5m
        spec:
          shell: Bash
          onDelegate: true
          source:
            type: Inline
            spec:
              script: ./check-tenant.sh <+repeat.item>
          environmentVariables: []
          outputVariables: []
```

7.7 Failure behavior in loops

When one iteration fails:

- Other **running** iterations continue unless your failure strategy aborts them.
- **Not yet started** iterations (due to `maxConcurrency`) start or not depending on the failure strategy applied to the stage or step group.
- The aggregate node (the parent) fails if any child fails, unless the failure strategy is `MarkAsSuccess` or the failure is ignored.

The relevant failure strategy actions are `Abort`, `StageRollback`, `PipelineRollback`, `Retry`, `ManualIntervention`, `MarkAsSuccess`, and `Ignore`. Chapter 12 covers them in detail. For deployments to multiple regions, a stage-level `StageRollback` rolls back the failing iteration, while `PipelineRollback` rolls back every completed deployment stage in the run, which may or may not be what you want.

7.7.1 Design rule: fail fast for the fleet

For a production fleet, decide up front:

Policy	Strategy
One bad region should stop all others from starting.	`maxConcurrency: 1` and a failure strategy that aborts or fails the pipeline.
All regions should attempt independently.	Higher concurrency, `Ignore` or continue-on-failure and an aggregation step at the end.
Wave-based.	Repeat with `maxConcurrency: 1` plus verification inside the iteration.

7.8 Combining matrix with services and environments

You can combine axes with `serviceRef` and `environmentRef` expressions to deploy services to environments in a single stage definition:

```
- stage:
    name: deploy_matrix
    identifier: deploy_matrix
    type: Deployment
    strategy:
      matrix:
        service: [orders_api, orders_worker]
        env: [staging]
        maxConcurrency: 2
    spec:
```

```
      deploymentType: Kubernetes
      service:
        serviceRef: <+matrix.service>
        serviceInputs: <+input>
      environment:
        environmentRef: <+matrix.env>
        deployToAll: false
        infrastructureDefinitions:
          - identifier: <+matrix.env>_cluster
```

Be aware that runtime inputs for each service (such as the artifact tag) multiply by the number of services. For multi-service deployments the dedicated `services` and `environments` constructs in Chapter 8 often give a cleaner experience than a raw matrix, because Harness understands them as service and environment collections and surfaces inputs per service.

7.9 Matrix versus multi-service

Aspect	`strategy.matrix` with service axis	`spec.services.values`
Runtime inputs per service	Awkward; inputs are per stage definition.	First-class; each service has its own `serviceInputs`.
Per-service dashboards	Works; each iteration is a deployment.	Works.
Parallel and sequential control	`maxConcurrency`	`metadata.parallel`
Combine with environments	Another axis.	`spec.environments.values` or environment groups.
Best for	Non-service axes (region, tenant, shard)	Services and environments

Use matrix for axes Harness does not model as first-class (tenant, shard, platform). Use the `services` and `environments` constructs for services and environments.

7.10 Practical limits and good behavior

- Large matrices create many nodes. Thousands of iterations make the execution view and API slow. Keep matrices in the tens, not thousands. For very large fan-out, use repeat with partitions, or dispatch work to another system and poll.
- Each iteration consumes delegate capacity. A matrix of fifty parallel deploys on a small delegate pool can saturate task slots and cause timeouts that look like deployment failures. Increase `maxConcurrency` carefully and scale delegates first.
- Keep iteration work idempotent. Retries and manual reruns must be safe.
- Prefer explicit node names so failures are legible in notifications.

7.11 Worked example: three-wave production rollout

This example deploys to five regions in three waves with verification in each wave, using repeat over wave definitions via a step group:

```
- stage:
    name: prod_waves
    identifier: prod_waves
    type: Deployment
    strategy:
      matrix:
        wave: [wave1, wave2, wave3]
        maxConcurrency: 1
    spec:
      deploymentType: Kubernetes
      service:
        serviceRef: orders_api
      environment:
        environmentRef: <+ <+matrix.wave> == "wave1" ? "prod_eu" : <+ <+matrix.wave> == "wave2" ? "prod_us" : "prod_apac" >>
        deployToAll: false
        infrastructureDefinitions:
          - identifier: cluster_main
      execution:
        steps:
          - step:
```

```
            type: K8sRollingDeploy
            name: Rollout <+matrix.wave>
            identifier: rollout
            timeout: 15m
            spec:
              skipDryRun: false
              pruningEnabled: false
        - step:
            type: Verify
            name: Verify <+matrix.wave>
            identifier: verify
            timeout: 30m
            spec:
              type: Rolling
              monitoredService:
                type: Default
                spec: {}
              spec:
                sensitivity: MEDIUM
                duration: 10m
                failOnNoAnalysis: false
      rollbackSteps:
        - step:
            type: K8sRollingRollback
            name: Rollback
            identifier: rollback
            timeout: 15m
            spec:
              pruningEnabled: false
  failureStrategies:
    - onFailure:
        errors:
          - AllErrors
        action:
          type: StageRollback
```

With `maxConcurrency: 1`, waves run in sequence. A failure in wave one stops the process before wave two begins. Nested ternary expressions work but are hard to read. For many waves, prefer a lookup in an environment variable or a small JSON map from an Http step.

8. Multi-Service and Multi-Environment Deployments

8.1 The problem space

Real releases are rarely a single service to a single cluster. A product release might include an API, a worker, and a web frontend, rolled out to staging and then to several production regions. Harness models this directly: a deployment stage can target a **list of services**, a **list of environments**, or an **environment group**, and the platform runs the cross-product as individual deployments grouped under one stage.

```
One stage definition
       |
       |  services: [orders_api, orders_worker, orders_web]
       |  environments: [prod_eu, prod_us]
       v
6 deployments (3 services x 2 environments), each with its own
inputs, logs, rollback, and status, shown under one stage.
```

This gives release engineers one place to look at a release, while keeping per-deployment isolation.

8.2 Multiple services in one stage

8.2.1 YAML

```yaml
- stage:
    name: deploy_services
    identifier: deploy_services
    type: Deployment
    spec:
      deploymentType: Kubernetes
      services:
        values:
          - serviceRef: orders_api
            serviceInputs:
              serviceDefinition:
                type: Kubernetes
                spec:
                  artifacts:
                    primary:
                      primaryArtifactRef: <+input>
                      sources: <+input>
          - serviceRef: orders_worker
            serviceInputs:
              serviceDefinition:
                type: Kubernetes
                spec:
                  artifacts:
                    primary:
                      primaryArtifactRef: <+input>
                      sources: <+input>
          - serviceRef: orders_web
            serviceInputs:
              serviceDefinition:
                type: Kubernetes
                spec:
                  artifacts:
                    primary:
                      primaryArtifactRef: <+input>
                      sources: <+input>
        metadata:
          parallel: true
      environment:
        environmentRef: staging
```

```
      deployToAll: false
      infrastructureDefinitions:
        - identifier: staging_cluster
    execution:
      steps:
        - step:
            name: Rollout
            identifier: rollout
            type: K8sRollingDeploy
            timeout: 10m
            spec:
              skipDryRun: false
              pruningEnabled: false
      rollbackSteps:
        - step:
            name: Rollback
            identifier: rollback
            type: K8sRollingRollback
            timeout: 10m
            spec:
              pruningEnabled: false
  failureStrategies:
    - onFailure:
        errors:
          - AllErrors
        action:
          type: StageRollback
```

The `execution` section is written **once**. Harness applies it to each service. Because the steps reference `<+service.name>` and `<+artifact.image>` implicitly (through the manifest rendering), each deployment uses its own service's manifests and artifact.

8.2.2 Parallel versus sequential

`metadata.parallel: true` deploys all services concurrently. `false` deploys them one after another in the listed order. Choose based on dependency:

Situation	Setting
Independent services	`true`, faster releases.
Strict order (database proxy before app)	`false` and order the list.
Constrained delegate capacity	`false`, or fewer parallel services using several stages.

8.2.3 Per-service inputs

Each entry has its own `serviceInputs`. When the pipeline runs, the form shows a section per service, so you can pick `1.4.2` for the API and `2.0.0` for the worker. In an input set, supply them per service.

```
services:
  values:
    - serviceRef: orders_api
      serviceInputs:
        serviceDefinition:
          type: Kubernetes
          spec:
            artifacts:
              primary:
                primaryArtifactRef: main_image
                sources:
                  - identifier: main_image
                    type: DockerRegistry
                    spec:
                      tag: 1.4.2
```

8.2.4 Runtime selection of the services

You can leave the list itself open:

```
services:
  values: <+input>
  metadata:
    parallel: true
```

The operator then picks which services to deploy at run time. This is the pattern for "hotfix only the worker".

8.3 Multiple environments in one stage

```
spec:
  deploymentType: Kubernetes
  service:
    serviceRef: orders_api
    serviceInputs:
      serviceDefinition:
        type: Kubernetes
        spec:
          artifacts:
            primary:
              primaryArtifactRef: <+input>
              sources: <+input>
  environments:
    values:
      - environmentRef: prod_eu
        deployToAll: false
        infrastructureDefinitions:
          - identifier: prod_eu_cluster
      - environmentRef: prod_us
        deployToAll: false
        infrastructureDefinitions:
          - identifier: prod_us_cluster
    metadata:
      parallel: true
```

The same service is deployed to each environment. Each environment's variables and overrides apply, so replicas, domain, and values files differ per region without any change to the pipeline.

8.3.1 `deployToAll`

`deployToAll: true` on an environment deploys to **every** infrastructure definition in it. This is how you implement "deploy to all clusters in this environment" when an environment represents a fleet of clusters. When `false`, list the specific infrastructure identifiers.

8.4 Multi-service by multi-environment

You can use both `services` and `environments` in one stage. Harness runs the cross-product.

```
spec:
  deploymentType: Kubernetes
  services:
    values:
      - serviceRef: orders_api
      - serviceRef: orders_worker
    metadata:
      parallel: true
  environments:
    values:
      - environmentRef: prod_eu
        deployToAll: true
      - environmentRef: prod_us
        deployToAll: true
```

```
    metadata:
      parallel: true
```

That is four deployments: each service to each environment. Parallelism flags apply independently: service parallelism and environment parallelism. A common production configuration is `environments.metadata.parallel: false` (region by region) with `services.metadata.parallel: true` (all services in a region together).

> **Region-by-region, services together**
>
> For fleet safety, deploy one region fully (all services in parallel) before the next region starts. Set `environments.metadata.parallel: false`. Add a verification step after rollout so a bad release stops at the first region.

8.5 Filters and environment groups

8.5.1 Environment groups

Reference a group instead of listing environments:

```
spec:
  deploymentType: Kubernetes
  service:
    serviceRef: orders_api
  environmentGroup:
    envGroupRef: production_regions
    deployToAll: true
    metadata:
      parallel: false
```

`deployToAll: true` includes every environment in the group, now and in the future. When you add `prod_apac` to the group, the pipeline picks it up on the next run with no edits. Make that a deliberate governance decision because it changes what existing pipelines deploy to.

8.5.2 Filters by tag

Environments and infrastructure definitions carry tags. Filters let you target the matching subset:

```
environments:
  filters:
    - entities:
        - environments
      type: tags
      spec:
        matchType: any
        tags:
          tier: prod
          region: eu
  metadata:
    parallel: true
```

`matchType` can be `any` or `all`. Filtering lets you write "all production EU environments" without naming them. Filters work best when the tagging convention is enforced, ideally through policy, because an untagged environment silently drops out of the selection.

> **Filter syntax**
>
> The filter structure above reflects the documented shape at the time of writing. Create a filter once in the UI (Environment section of the stage) and copy the generated YAML into your pipeline repository to get the exact schema for your release.

8.6 Expressions in multi-service stages

Because one execution definition fans out to many deployments, use identity expressions rather than hard-coded names:

Expression	Resolves to
`<+service.name>`	The service of the current deployment.
`<+service.identifier>`	Its identifier.
`<+env.name>`, `<+env.identifier>`	The environment of the current deployment.
`<+infra.name>`, `<+infra.namespace>`	The infrastructure of the current deployment.
`<+serviceVariables.x>`	Effective variable value for this service in this environment.

```
- step:
    type: ShellScript
    name: Announce
    identifier: announce
    timeout: 2m
    spec:
      shell: Bash
      onDelegate: true
      source:
        type: Inline
        spec:
          script: |
            echo "Deploying <+service.name> <+artifact.tag> to <+env.name>/<+infra.name>"
      environmentVariables: []
      outputVariables: []
```

Each of the cross-product deployments prints its own line.

8.7 Services with different shapes in one stage

All services in one stage share the `execution` section. That implies they share the **deployment type** (all Kubernetes) and a compatible set of steps. A rolling deploy step suits services that all support rolling. A mix of rolling and blue-green services does not fit in one stage. Group services by strategy into separate stages:

```
Stage 1: stateless services -> rolling (services: api, web, worker)
Stage 2: stateful service   -> blue-green (service: ledger)
```

Use pipeline ordering and barriers to coordinate when stages depend on each other.

8.8 Multi-service with approvals and verification

Where do approvals and verification fit relative to fan-out? Two options:

1. **Inside the fan-out execution.** The step runs once per service-environment deployment. Ten deployments means ten approvals. Rarely wanted.
2. **Outside, in separate stages before and after the deployment stage.** One approval covers the entire release. This is the usual choice.

Verification (Chapter 10) is different: it is naturally per service, because each service has its own metrics. Put Verify steps inside the multi-service execution and let it run per deployment.

```
Stage: approve release (one ServiceNow approval)
Stage: deploy services (fan-out) -> rollout -> verify (per service)
Stage: post-release tasks (one notification)
```

8.9 Rollback semantics in multi-deployment stages

Each cross-product deployment has its own rollback steps execution. If one deployment fails:

- With a stage-level `StageRollback`, the failing deployment runs rollback steps. The behavior for sibling deployments depends on your parallelism and failure strategy configuration. Siblings that already succeeded are not automatically rolled back by `StageRollback` for

the failing deployment alone.

- With `PipelineRollback`, Harness rolls back stages that completed successfully, in reverse order, which gives an all-or-nothing release when your design wants atomic behavior.

Understand and test the exact behavior in a sandbox before using multi-service stages for production. Design explicit policies:

Desired outcome	Approach
Atomic release across services	`PipelineRollback` on failure; or stages ordered so earlier stages can be rolled back.
Independent services	`StageRollback` per deployment; accept partial release; use dashboards to track.
Stop the line	`Abort` on failure to prevent further deployments, then decide manually.

> **Partial releases are valid states**
>
> With independent services, a partial release means some services run the new version and some the old. Make sure service-to-service APIs are compatible across adjacent versions, or the release strategy itself causes outages.

8.10 Service propagation across stages

When you deploy the same service through several stages (dev, then staging, then production), you do not need to re-select the service in each stage. Stages can **propagate** a service from a previous stage:

```
- stage:
    name: deploy_staging
    identifier: deploy_staging
    type: Deployment
    spec:
      deploymentType: Kubernetes
      service:
        useFromStage:
          stage: deploy_dev
      environment:
        environmentRef: staging
        deployToAll: false
        infrastructureDefinitions:
          - identifier: staging_cluster
      execution:
        steps:
          - step:
              type: K8sRollingDeploy
              name: Rollout
              identifier: rollout
              timeout: 10m
              spec:
                skipDryRun: false
                pruningEnabled: false
```

Propagation guarantees that the **same service and artifact** are deployed in each stage and the operator provides inputs once. It also reduces approval risk: what was verified in staging is by construction what goes to production.

Service propagation works for single-service stages. For multi-service stages, propagate by pipeline structure and input sets, because `useFromStage` refers to a stage's single service.

8.11 Environment propagation and chained pipelines

Environments and infrastructure can also be propagated from a prior stage (`useFromStage` under `environment`). This is helpful in a "deploy then run a second set of steps to the same target" design.

For very large systems, break a mega-release into pipelines and chain them. A parent pipeline calls child pipelines through **Pipeline Chaining** stages, passing inputs and reading outputs. This keeps each pipeline understandable and independently runnable, while the parent coordinates the order. Use chaining when:

- Teams own different services and their pipelines.

- You need to run a child with different permissions.
- The release train consists of independently useful pipelines.

8.12 Performance and capacity

Concern	Guidance
Delegate task limits	Each parallel deployment uses delegate task slots. Estimate peak parallelism and size the delegate pool accordingly.
API rate limits	Many simultaneous manifest fetches from Git can hit provider rate limits. Use fewer parallel services or a Git cache close to the delegate.
Execution graph size	A 10 by 10 cross-product renders 100 deployment nodes. Keep it manageable.
Blast radius	Parallel production deployments to all regions at once maximize speed and blast radius. Prefer region-by-region with verification.

8.13 Checklist for multi-service releases

1. Decide atomic versus independent semantics up front.
2. Group services by deployment strategy into stages.
3. Choose parallelism per axis deliberately.
4. Put approvals outside the fan-out, verification inside.
5. Use environment groups or tag filters, with enforced tagging.
6. Provide per-service inputs through input sets stored with the pipeline.
7. Test failure of one service in a sandbox and record the observed behavior.

9. Approvals and ServiceNow Integration

9.1 Approval types in Harness CD

Harness supports several approval mechanisms, and larger organizations typically mix them.

Approval	What decides	Typical use
Harness Approval	Users or user groups approve inside Harness.	Lightweight sign-off, QA lead, release manager.
Jira Approval	Fields on a Jira issue match criteria.	Teams that track releases in Jira.
ServiceNow Approval	Fields on a ServiceNow ticket match criteria.	ITIL change management, CAB.
Custom Approval	A script's result determines approval.	Internal systems with no native integration.

Every approval step is a **gate**. The pipeline pauses at it, polls or waits, and either proceeds, fails, or times out.

9.2 Harness native approval

```
- step:
    type: HarnessApproval
    name: Release Manager Approval
    identifier: release_manager_approval
    timeout: 1d
    spec:
      approvalMessage: |-
        Approve production rollout of <+pipeline.variables.releaseName>.
        Environment: <+env.name>
        Image: <+artifact.image>
      includePipelineExecutionHistory: true
      approvers:
        userGroups:
          - release_managers
        minimumCount: 1
        disallowPipelineExecutor: true
      approverInputs:
        - name: rollbackWindow
          value: 60
      isAutoRejectEnabled: false
```

Notable settings:

Setting	Meaning
`userGroups`	Who may approve. Use groups, not individuals.
`minimumCount`	How many approvals are required. Two for four-eyes principle.
`disallowPipelineExecutor`	If `true`, the person who started the run cannot approve it. This is the separation-of-duties switch.
`approverInputs`	Values the approver can supply, usable later as `<+pipeline.stages.stage.spec.execution.steps.approval.output.approverInputs.rollbackWindow>`.
`includePipelineExecutionHistory`	Shows prior stage results to the approver.
`timeout`	Maximum wait, for example `1d`. On expiry the step fails (or follows the failure strategy).

> **Separation of duties**
>
> For production, set `disallowPipelineExecutor: true` and `minimumCount: 2` with a user group that is separate from the delivery team. Auditors look for exactly this.

9.2.1 Auto-approval conditions

For low-risk changes you may want approvals only when needed. Use conditional execution on the approval step so it only runs for production or for major version changes:

```
when:
  stageStatus: Success
  condition: <+ <+env.type> == "Production" >
```

If the condition is false, the step is skipped and the pipeline continues.

9.3 Why ServiceNow

In regulated environments, a production deployment must correspond to an approved **change request**. The approval is recorded in the IT service management (ITSM) system, not in the CD tool. Harness integrates in three ways:

1. **Create** a ticket (change request, incident, task) from the pipeline.
2. **Update** a ticket (fields, state transitions) from the pipeline.
3. **Approve**: pause the pipeline until the ticket meets approval criteria, and fail it if it meets rejection criteria.

The ServiceNow steps are `ServiceNowCreate`, `ServiceNowUpdate`, `ServiceNowApproval`, and `ServiceNowImportSet`.

```
  Pipeline start
       |
  ServiceNowCreate (change request, state: New)
       |
  ServiceNowApproval (wait until state = Implement, within change window)
       |
  Deploy + Verify
       |
  ServiceNowUpdate (state: Review, close notes)
```

9.4 ServiceNow connector

A ServiceNow connector stores the instance URL and credentials. Prefer a **dedicated integration user** with minimum roles, and keep the password in a Harness secret.

```
connector:
  name: servicenow_prod
  identifier: servicenow_prod
  type: ServiceNow
  orgIdentifier: platform
  projectIdentifier: payments
  spec:
    serviceNowUrl: https://acme.service-now.com
    auth:
      type: UsernamePassword
      spec:
        username: svc_harness
        passwordRef: account.servicenow_password
    delegateSelectors:
      - prod-eu
```

Other authentication types, such as OAuth-style or ADFS and client certificate flows, are available depending on your ServiceNow setup. Choose the one your security team standardizes on.

9.4.1 ServiceNow permissions

The integration user needs enough access to read and write the table the pipeline uses (for example `change_request`), read the fields used in criteria, and query the dictionary for field metadata, because Harness lists available fields when you configure steps in the UI. Over-granting risks accidental ticket modification. Under-granting produces confusing errors such as empty field lists.

Error	Likely cause
401 or 403 from ServiceNow	Wrong credentials or missing roles.
Fields not listed in UI	User cannot read `sys_dictionary`.
Timeouts connecting	Delegate cannot reach the instance; check proxy and allowlists.
SSL errors	Private CA; install the certificate on the delegate.

9.5 Creating a change request

```
- step:
    type: ServiceNowCreate
    name: Create Change Request
    identifier: create_change_request
    timeout: 5m
    spec:
      connectorRef: servicenow_prod
      ticketType: change_request
      fields:
        - name: short_description
          value: "Deploy <+service.name> <+artifact.tag> to <+env.name>"
        - name: description
          value: |-
            Automated change from Harness pipeline <+pipeline.name>.
            Execution: <+pipeline.executionUrl>
        - name: category
          value: Software
        - name: priority
          value: "3"
        - name: assignment_group
          value: Release Engineering
      createType: Normal
```

9.5.1 Create types

`createType`	Description
`Normal`	Specify fields directly.
`Standard`	Use a ServiceNow **standard change template**. Pre-approved changes follow a fast path.
`Form`	Create using the ServiceNow form representation.

Standard changes are valuable for low-risk, repeatable deployments, because the template pre-authorizes the change class. A pipeline that creates a standard change with the right template can proceed without waiting for a CAB.

9.5.2 Output variables from the create step

The created ticket's identifiers become outputs:

Expression	Value
`<+execution.steps.create_change_request.ticket.ticketNumber>`	The change number, for example `CHG0034567`.
`<+execution.steps.create_change_request.ticket.ticketUrl>`	Link to the ticket.
`<+execution.steps.create_change_request.ticket.fields.state>`	Field value as returned.

The exact output path differs slightly by version. The execution *Output* tab of the step lists all available keys. Capture the ticket number in a pipeline-level or stage-level variable if later stages need it.

9.6 Approving on a ticket

```yaml
- step:
    type: ServiceNowApproval
    name: Wait For Change Approval
    identifier: wait_for_change_approval
    timeout: 2d
    spec:
      connectorRef: servicenow_prod
      ticketType: change_request
      ticketNumber: <+execution.steps.create_change_request.ticket.ticketNumber>
      retryInterval: 1m
      approvalCriteria:
        type: KeyValues
        spec:
          matchAnyCondition: false
          conditions:
            - key: state
              operator: equals
              value: Implement
            - key: approval
              operator: equals
              value: approved
      rejectionCriteria:
        type: KeyValues
        spec:
          matchAnyCondition: true
          conditions:
            - key: state
              operator: equals
              value: Canceled
            - key: approval
              operator: equals
              value: rejected
```

The step polls the ticket every `retryInterval`. It **succeeds** when the approval criteria are satisfied, **fails** when the rejection criteria are satisfied, and otherwise keeps waiting until the timeout.

9.6.1 Criteria types and operators

Criteria are defined as one of two types:

Type	Use
`KeyValues`	A list of field conditions with an `operator`.
`Jexl`	A JEXL expression evaluated against ticket fields, for complex logic.

Operators for key-value conditions include `equals`, `not equals`, `in`, and `not in`. The `in` operator takes a comma-separated list, for example approve when `state` is in `Implement,Scheduled`.

`matchAnyCondition` controls combination: `true` means any single condition is enough, `false` means all conditions must hold.

A JEXL approval criteria example, using field names from the ticket:

```yaml
approvalCriteria:
  type: Jexl
  spec:
    expression: <+ <+ticket.state.displayValue> == "Implement" && <+ticket.approval.value> == "approved" >
```

JEXL gives you arithmetic, string checks, and field comparisons not possible with the plain key-value form. The accessor names above (`ticket.state.displayValue` and similar) illustrate the idea. The exact object shape that Harness exposes for ticket fields is shown in the step's input and output tabs of a previous execution, so copy accessors from a real run rather than from this guide. Test the expression in a sandbox against a real ticket, since field internal values and display values differ in ServiceNow (for example, the numeric state code versus the label).

> **Internal value versus display value**
>
> ServiceNow fields commonly have an internal value (such as `-1` for `Implement` in some states models) and a display label. Key-value conditions typically match against what the Harness UI offers in its dropdown. If a condition never matches although the ticket looks right, compare against the other representation and check your instance's state model.

9.6.2 Change windows

A **change window** ensures the deployment happens only inside the approved window recorded on the ticket. The step reads two date-time fields from the ticket and waits for the window to open, and fails if the window is already closed.

```
spec:
  connectorRef: servicenow_prod
  ticketType: change_request
  ticketNumber: <+pipeline.variables.changeNumber>
  approvalCriteria:
    type: KeyValues
    spec:
      matchAnyCondition: false
      conditions:
        - key: state
          operator: equals
          value: Implement
  rejectionCriteria:
    type: KeyValues
    spec:
      matchAnyCondition: true
      conditions:
        - key: state
          operator: equals
          value: Canceled
  changeWindow:
    startField: start_date
    endField: end_date
```

Behavior:

- **Approved and inside the window**: the step succeeds immediately.
- **Approved but window not yet open**: the step waits until the start time, then succeeds.
- **Window closed**: the step fails. Create a new window on the ticket before re-running.

Fields commonly used are the planned start and end dates, or the work start and end dates, depending on your process. Confirm the field names in your instance. Time zones matter: ServiceNow stores UTC internally, while users edit in their profile time zone. Harness evaluates the stored value.

9.6.3 Polling cost and timeouts

Long approvals poll the instance for days. Choose `retryInterval` sensibly (a minute is typical) and remember the pipeline execution is waiting without consuming a delegate task continuously, but it does produce API calls. A ServiceNow instance that rate-limits integration users can throttle you. For a change that waits two days, a five-minute interval is plenty.

9.7 Updating tickets

```
- step:
    type: ServiceNowUpdate
    name: Move Change To Review
    identifier: move_change_to_review
    timeout: 5m
    spec:
      connectorRef: servicenow_prod
      ticketType: change_request
      ticketNumber: <+execution.steps.create_change_request.ticket.ticketNumber>
      useServiceNowTemplate: false
      fields:
```

```
        - name: state
          value: Review
        - name: close_code
          value: successful
        - name: close_notes
          value: |-
            Deployed <+artifact.image> to <+env.name>.
            Pipeline execution: <+pipeline.executionUrl>
```

Update steps belong in two places:

1. **On success**, after verification completes, to move the ticket to review or closed.
2. **On failure**, in a failure-strategy path or a rollback step, to record that the change failed and was rolled back.

9.7.1 Updating on failure

Use a stage-level failure strategy that runs a step group before rollback, or place an update step in the `rollbackSteps` so the ticket always reflects what happened:

```
rollbackSteps:
  - step:
      type: K8sRollingRollback
      name: Rollback
      identifier: rollback
      timeout: 10m
      spec:
        pruningEnabled: false
  - step:
      type: ServiceNowUpdate
      name: Record Rollback
      identifier: record_rollback
      timeout: 5m
      spec:
        connectorRef: servicenow_prod
        ticketType: change_request
        ticketNumber: <+pipeline.variables.changeNumber>
        useServiceNowTemplate: false
        fields:
          - name: state
            value: Review
          - name: close_code
            value: unsuccessful
          - name: close_notes
            value: Automatic rollback after failed verification.
```

An audit trail with a correct ending state saves time during post-incident review.

9.8 Import sets

`ServiceNowImportSet` posts data to a ServiceNow **import set** staging table, where transform maps then load it into target tables such as a CMDB. Use it to record deployment facts, like service, version, environment, and timestamps, in your CMDB.

```
- step:
    type: ServiceNowImportSet
    name: Record Deployment In CMDB
    identifier: record_deployment_cmdb
    timeout: 5m
    spec:
      connectorRef: servicenow_prod
      stagingTableName: u_harness_deployments
      importData:
        type: JsonBody
        spec:
          jsonBody: |-
            {
              "service": "<+service.name>",
```

```
            "version": "<+artifact.tag>",
            "environment": "<+env.name>",
            "pipeline_execution": "<+pipeline.executionUrl>"
          }
```

It requires a prepared staging table and transform map on the ServiceNow side. Work with the ServiceNow administrators; the pipeline side is the easy part.

9.9 End-to-end change-managed stage

The full lifecycle in one pipeline:

```yaml
pipeline:
  name: orders-prod-change-managed
  identifier: orders_prod_change_managed
  projectIdentifier: payments
  orgIdentifier: platform
  variables:
    - name: changeNumber
      type: String
      value: <+input>.regex(^CHG[0-9]{7}$)
  stages:
    - stage:
        name: change_gate
        identifier: change_gate
        type: Approval
        spec:
          execution:
            steps:
              - step:
                  type: ServiceNowApproval
                  name: Change Approved
                  identifier: change_approved
                  timeout: 2d
                  spec:
                    connectorRef: servicenow_prod
                    ticketType: change_request
                    ticketNumber: <+pipeline.variables.changeNumber>
                    retryInterval: 2m
                    approvalCriteria:
                      type: KeyValues
                      spec:
                        matchAnyCondition: false
                        conditions:
                          - key: state
                            operator: equals
                            value: Implement
                    rejectionCriteria:
                      type: KeyValues
                      spec:
                        matchAnyCondition: true
                        conditions:
                          - key: state
                            operator: equals
                            value: Canceled
                    changeWindow:
                      startField: start_date
                      endField: end_date
        failureStrategies:
          - onFailure:
              errors:
                - AllErrors
              action:
                type: Abort
    - stage:
        name: deploy_prod
        identifier: deploy_prod
```

```
      type: Deployment
      spec:
        deploymentType: Kubernetes
        service:
          serviceRef: orders_api
          serviceInputs: <+input>
        environment:
          environmentRef: prod_eu
          deployToAll: false
          infrastructureDefinitions:
            - identifier: prod_eu_cluster
        execution:
          steps:
            - step:
                type: K8sRollingDeploy
                name: Rollout
                identifier: rollout
                timeout: 15m
                spec:
                  skipDryRun: false
                  pruningEnabled: false
          rollbackSteps:
            - step:
                type: K8sRollingRollback
                name: Rollback
                identifier: rollback
                timeout: 15m
                spec:
                  pruningEnabled: false
      failureStrategies:
        - onFailure:
            errors:
              - AllErrors
            action:
              type: StageRollback
  - stage:
      name: close_change
      identifier: close_change
      type: Approval
      spec:
        execution:
          steps:
            - step:
                type: ServiceNowUpdate
                name: Close Change
                identifier: close_change_step
                timeout: 5m
                spec:
                  connectorRef: servicenow_prod
                  ticketType: change_request
                  ticketNumber: <+pipeline.variables.changeNumber>
                  useServiceNowTemplate: false
                  fields:
                    - name: state
                      value: Review
                    - name: close_code
                      value: successful
```

The first stage is an `Approval` stage type, a lightweight stage that holds approval steps without needing a service or environment. Putting the approval in its own stage keeps the change gate visible and reusable, and means the deployment stage has no waiting logic.

9.9.1 Pre-existing ticket versus pipeline-created ticket

Model	Description	Trade-off
Ticket first.	The requester creates the change in ServiceNow, then runs the pipeline with the change number.	Matches existing ITIL practice. Needs input validation (regex) and a check that the ticket covers this service and environment.

Model	Description	Trade-off
Pipeline creates.	The pipeline creates the change, then waits for approvals.	Faster for teams and standard changes. Requires automation of risk and assignment fields, and a CAB process that handles automated submissions.

For ticket-first flows, add a validation step that reads the ticket (through an Http step or a ServiceNow update that touches nothing) and asserts that its configuration item or description matches the service being deployed. It prevents the use of an unrelated approved ticket as a rubber stamp.

9.10 Approval timeouts, rejection, and expiry

Event	Behavior
Approval criteria met	Step succeeds, pipeline continues.
Rejection criteria met	Step fails. Failure strategy applies.
Timeout reached	Step fails with a timeout error.
Pipeline aborted during wait	Step is aborted.
Ticket deleted or number invalid	Step fails with an error from ServiceNow.

For long-running approvals, add a notification rule for the approval step, covering pending, expired, and failed events, to Slack or email so humans learn of waiting approvals. A forgotten approval means a stalled release.

9.11 Approval design patterns

Pattern	Description
Single gate before production	One approval stage ahead of the production deployment stage. Simple and common.
Two-person rule	Harness Approval with `minimumCount: 2`, plus ServiceNow change. Meets strict audit controls.
Risk-based gating	Conditional execution selects approval type by risk label or environment. Low risk uses a standard change, high risk waits for CAB.
Post-deployment attestation	After verification, update the ticket. Humans do not approve, but the ticket reflects what happened.
Emergency path	A separate pipeline or an input flag that creates an emergency change with a mandatory retrospective step.

9.12 Troubleshooting approvals

Symptom	Check
ServiceNow Approval never completes.	The ticket field values versus criteria (internal versus display), `matchAnyCondition`, and whether the ticket is the right number.
Immediately fails with change window error.	The window is closed. Verify field names and time zone.
Cannot select fields in UI.	Integration user lacks dictionary read, or the ticket type is wrong.
Approval notified but not visible to approver.	Approver is not in the configured user group or is the executor with `disallowPipelineExecutor: true`.
Approval stage consumes timeouts.	Set a realistic `timeout` and failure strategy; use notifications.

10. Continuous Verification with Datadog

10.1 What continuous verification does

A successful rollout means Pods became Ready. It does not mean the release is healthy. Error rates, latency, saturation, and log patterns can regress while every health check still passes. **Continuous Verification (CV)** automates the judgment that a human would otherwise make by staring at dashboards after a deploy.

In Harness, CV is performed by the **Verify** step in a deployment stage. The step:

1. Waits for the configured **duration** while collecting metrics and logs from your **health sources** (here, Datadog).
2. Compares the data from the new version against a **baseline** (the previous version, a prior time window, or pre-deployment data), using machine learning for time series and log clustering.
3. Produces a **risk score** and a verdict, and fails the step if the risk exceeds the configured **sensitivity**.
4. Lets the stage's **failure strategy** decide what happens next, typically an automatic rollback.

```
Deploy ----> [ Verify step: collect for N minutes ] ----> analyze ----> pass: continue
   |              metrics + logs from Datadog                |
   |                                                          +-> fail: failure strategy
   |                                                              (rollback / pause / abort)
   +--- new Pods start taking traffic
```

10.2 Building blocks

Concept	Description
Connector	Credentials for Datadog (API key and application key).
Health source	A specific data source: Datadog metrics, Datadog logs. Several can be attached to one service.
Monitored service	A Harness entity representing the service plus environment, with health sources attached. The Verify step analyses a monitored service.
Verify step	The pipeline step that triggers analysis.
Sensitivity	How readily deviations are flagged: `LOW`, `MEDIUM`, `HIGH`.
Duration	How long to analyze after deployment: for example `5m`, `10m`, `15m`, `30m`.
Verification type	`Auto`, `Rolling`, `Canary`, `BlueGreen`, `LoadTest`.

10.2.1 Verification types

Type	Baseline	Typical strategy
`Rolling`	Pre-deployment behavior and the previous version's data.	Rolling deployment.
`Canary`	Stable Pods (control) compared with canary Pods (test), at the same time.	Canary deployment.
`BlueGreen`	The old environment compared with the new one.	Blue-green.
`LoadTest`	A previous load test run with the same shape.	Pre-production load tests.
`Auto`	Harness chooses based on the deployment strategy.	Reasonable default for standard strategies.

Canary verification is the most reliable method because the baseline and the test data are collected simultaneously on the same traffic, which cancels time-of-day effects. Rolling verification is weaker for services with strong daily patterns, because the comparison uses data from a different time.

10.3 Datadog connector

```yaml
connector:
  name: datadog_main
```

```
  identifier: datadog_main
  type: Datadog
  orgIdentifier: platform
  projectIdentifier: payments
  spec:
    url: https://api.datadoghq.com/api/
    apiKeyRef: datadog_api_key
    applicationKeyRef: datadog_app_key
    delegateSelectors:
      - observability
```

Field	Notes
`url`	Match your Datadog site. For example `https://api.datadoghq.com/api/` (US1), `https://api.datadoghq.eu/api/` (EU), `https://api.us3.datadoghq.com/api/`, `https://api.us5.datadoghq.com/api/`, `https://api.ap1.datadoghq.com/api/`, or `https://api.ddog-gov.com/api/` for the US government site. A key from one site fails on another.
`apiKeyRef`	Secret holding the Datadog API key.
`applicationKeyRef`	Secret holding an application key. Application keys inherit the permissions of the user who created them, so create the key under a dedicated service account with read-only scopes.
`delegateSelectors`	The delegate performs the Datadog calls. It needs egress to the Datadog API endpoint.

> **Key hygiene**
>
> Rotate Datadog keys on a schedule. Store them only as Harness secrets (preferably in an external secret manager). A leaked application key lets someone read your telemetry, and potentially more depending on its scopes.

Use the connector's **Test** action. A successful test proves the delegate can reach Datadog and the keys are valid. It does not prove the keys have permission to read the metrics you will query.

10.4 Defining the health source

You can attach a health source in the monitored service UI, or define a monitored service as a template and let the Verify step use it. A health source for Datadog can contain metric definitions, log definitions, or both.

10.4.1 Metrics

For Datadog metrics you select or write queries. Each **metric definition** has:

Field	Meaning
Name and identifier	A label for the metric in analysis views.
Query	A Datadog metric query.
Group name	Logical grouping for display, for example `Latency` or `Errors`.
Metric path or dashboard	Alternatively select an existing Datadog dashboard widget as the source.
Service instance identifier	The tag that identifies the instance (Pod, host) that emitted the data. Needed for canary comparison and node-level analysis.
Risk category	Performance, errors, infrastructure, or custom.
Deviation direction	Alert on higher values, lower values, or both.
Include in verification	Whether this metric participates in deployment verification.

Query design

A good verification query is **scoped** to the service and environment under test, and **grouped by the instance identifier** so Harness can tell canary Pods from baseline Pods.

Examples of the kind of queries used (adapt tag names to your tagging scheme):

```
# Error rate as a ratio, per Pod
sum:trace.http.request.errors{service:orders-api,env:prod}.as_rate() by {pod_name}
```

```
# Request latency p95 from an APM distribution metric
p95:trace.http.request{service:orders-api,env:prod} by {pod_name}

# Container CPU
avg:kubernetes.cpu.usage.total{kube_deployment:orders-api,kube_namespace:orders} by {pod_name}

# Container memory
avg:kubernetes.memory.working_set{kube_deployment:orders-api,kube_namespace:orders} by {pod_name}
```

The `by {pod_name}` clause is what makes per-instance analysis possible. Without it, Harness sees one aggregated series and cannot separate the canary from the baseline.

> **Use Harness-provided placeholder when available**
>
> Harness supports a placeholder in Datadog queries for the service instance filter so a single query can be run for the instances under analysis (for example, the Pods in the canary set). Look for the filter placeholder hint in the metric definition dialog for your release and use it instead of hard-coding Pod names, which change on every deploy.

10.4.2 Logs

Datadog log health sources let Harness cluster log lines and detect new or anomalous error patterns after a deploy.

Field	Meaning
Query	A Datadog log search query, for example `service:orders-api env:prod status:error`.
Service instance identifier	The log attribute that carries the Pod or host name.
Index	Optionally restrict to a specific Datadog log index.

Log analysis highlights three classes of event:

Class	Meaning
Known	Patterns seen in the baseline. Not suspicious.
Unknown	New patterns that did not exist before. Candidates for regressions.
Unexpected frequency	Known patterns appearing much more often than before.

Unknown patterns are not automatically bad. A new log line announcing a new feature will look "unknown". Harness lets you mark events as expected, which feeds later runs and reduces noise.

10.4.3 Metric thresholds

Raw ML analysis can be too sensitive or not sensitive enough for a particular metric. **Metric thresholds** let you encode your operational rules:

Threshold type	Effect
Ignore	Do not factor this metric (or deviations below a threshold) into the verdict. Useful for known-noisy metrics.
Fail fast	Fail the verification immediately when a metric crosses a configured limit, without waiting for the full duration.

Fail-fast thresholds are a strong safety net: "if the error rate exceeds 5 percent at any point, fail now". They shorten the time bad releases stay exposed to users. Use ignore thresholds sparingly and document why, because they can mask real regressions.

10.5 The Verify step

```
- step:
    type: Verify
    name: Verify Canary
    identifier: verify_canary
    timeout: 2h
    spec:
```

```
      type: Canary
      monitoredService:
        type: Default
        spec: {}
      spec:
        sensitivity: MEDIUM
        duration: 10m
        deploymentTag: <+serviceConfig.artifacts.primary.tag>
        failOnNoAnalysis: false
    failureStrategies:
      - onFailure:
          errors:
            - Verification
          action:
            type: StageRollback
```

10.5.1 Fields

Field	Meaning
`type`	One of `Auto`, `Rolling`, `Canary`, `BlueGreen`, `LoadTest`.
`monitoredService.type`	`Default` (derive the monitored service from the service and environment, which Harness creates and links), `Configured` (select an existing one), or `Template` (a monitored service template with input values).
`sensitivity`	`LOW`, `MEDIUM`, `HIGH`. Higher sensitivity flags smaller deviations.
`duration`	Verification window. Common values are 5, 10, 15, 30 minutes.
`deploymentTag`	For canary and rolling checks, the tag identifying the new version in telemetry. Used to select data by version where tags are available.
`failOnNoAnalysis`	If `true`, fail when no data was available to analyze.
`timeout`	Upper bound for the step including the wait for data. Make it comfortably larger than the duration.

10.5.2 Default monitored service

With `type: Default`, Harness derives a monitored service identity from the service and environment of the stage. This works when you have attached the health sources to the monitored service for that service and environment pair. A consequence: the monitored service and its health sources need to exist per **service and environment combination**. For many environments, use **monitored service templates** so you define the Datadog queries once and fill in environment-specific tags.

10.5.3 Monitored service templates

A template defines the health source structure with runtime inputs for values that differ (for example, `env` tag, namespace). In the Verify step, choose `type: Template` and provide the inputs, often as expressions:

```
monitoredService:
  type: Template
  spec:
    monitoredServiceTemplateRef: orders_datadog_template
    versionLabel: v1
    templateInputs:
      type: Application
      sources:
        healthSources:
          - identifier: dd_metrics
            type: Datadog
            spec:
              queries:
                - identifier: error_rate
                  query: "sum:trace.http.request.errors{service:orders-api,env:<+env.variables.ddEnv>}.as_rate() by {pod_name}"
```

The YAML shape of template inputs is generated by the UI; use the UI to create it once and copy the output. Writing it by hand is error-prone.

10.6 Verification across deployment strategies

10.6.1 Rolling

```
execution:
  steps:
    - step:
        type: K8sRollingDeploy
        name: Rollout
        identifier: rollout
        timeout: 10m
        spec:
          skipDryRun: false
          pruningEnabled: false
    - step:
        type: Verify
        name: Verify Rolling
        identifier: verify_rolling
        timeout: 1h
        spec:
          type: Rolling
          monitoredService:
            type: Default
            spec: {}
          spec:
            sensitivity: MEDIUM
            duration: 10m
            failOnNoAnalysis: false
        failureStrategies:
          - onFailure:
              errors:
                - Verification
              action:
                type: StageRollback
  rollbackSteps:
    - step:
        type: K8sRollingRollback
        name: Rollback
        identifier: rollback
        timeout: 10m
        spec:
          pruningEnabled: false
```

With rolling verification the new version has already replaced the old one by the time analysis starts. The harm window is the verification duration plus rollback time. Choose a shorter duration for high-traffic services where a few minutes provide enough samples.

10.6.2 Canary

Verify between canary deploy and canary delete, as in Chapter 5. Because only a fraction of traffic reaches the canary, the harm window is smaller. The cost: the canary needs enough traffic to produce statistically meaningful data. A canary of one Pod receiving a handful of requests per minute yields weak signals. Increase the canary size or the duration.

10.6.3 Blue-green

Run Verify after `K8sBlueGreenDeploy` and before the swap. Synthetic traffic or a test load generator typically drives the stage Service, because live traffic still goes to the old version. If there is no traffic on the stage side, there is nothing to verify. Alternatively, swap first and verify after, with an immediate swap-back on failure. That exposes users but tests real traffic.

10.7 Failure handling for verification

A Verify step failure has the error type `Verification`. You can attach step-level or stage-level failure strategies keyed on that error:

```
failureStrategies:
  - onFailure:
      errors:
```

```
        - Verification
      action:
        type: ManualIntervention
        spec:
          timeout: 30m
          onTimeout:
            action:
              type: StageRollback
```

`ManualIntervention` pauses the pipeline so a human can examine the analysis and choose: **Mark as Success**, **Rollback**, **Retry**, or **Abort**. If nobody responds within the timeout, the configured `onTimeout` action happens (above, a rollback).

Policy	When it fits
Automatic `StageRollback`	Mature signals, high trust in thresholds, strict SLOs.
`ManualIntervention` with rollback on timeout	Early adoption, to build trust in the analysis.
`Ignore` or `MarkAsSuccess`	Never for production. Use only in experiments.

> **Start in observation mode**
>
> For the first few weeks, run Verify with `ManualIntervention` so a human reviews every failed verdict. Tune queries, ignore thresholds, and sensitivity until failed verdicts are almost always real regressions. Then switch to automatic rollback.

10.8 Reading analysis results

The Verify step's execution view includes tabs for:

View	What it shows
Summary	Overall risk, duration, sensitivity, status.
Metrics	Per-metric time series with baseline and test overlays, risk per metric.
Logs	Clusters with category (known, unknown, unexpected frequency).
Nodes	Per-instance health (Pods) with risk levels.
Console	The step's raw logs and API interactions.

Risk is expressed as levels such as **Healthy**, **Observe**, and **Needs attention**, derived from the numeric score and the sensitivity. When a verification fails, start with the Metrics tab. Sort by risk, and look at the overlay. A one-pod outlier is a different problem from a fleet-wide shift.

10.9 Why verification returns no data

The most common problems are about data, not about Harness.

Symptom	Likely cause	Fix
"No analysis"	Queries return nothing for the window.	Run the query in Datadog Metrics Explorer for the same time range. Fix tags.
Only baseline, no test data	Instance identifier tag does not match the new Pods.	Group by the right tag (`pod_name`, `kube_pod`, `host`). Confirm the tag is present on the new version's metrics.
Canary and baseline look identical	Query is aggregated across all Pods.	Add `by {pod_name}`.
403 from Datadog	Application key lacks permission.	Use a key from a user or service account with read access to the metrics and logs.
Wrong site	Connector URL points at a different Datadog site.	Use the correct regional URL.
Verification passes during an outage	Too few samples; metrics are zero or missing for both sets.	Set `failOnNoAnalysis: true` for production. Add a minimum traffic check before Verify.

Symptom	Likely cause	Fix
Frequent false failures	Over-sensitive metric, noisy log patterns.	Lower sensitivity for noisy metrics, add ignore thresholds, mark expected log events.
Delay between deploy and data	Datadog ingestion delay.	Account for a one to two minute lag. Increase the duration or add a short wait before Verify.

10.9.1 Make the data available before the step

Datadog agents need to tag the new Pods consistently. Use Unified Service Tagging labels (`env`, `service`, `version`) on Pods:

```
metadata:
  labels:
    tags.datadoghq.com/env: <+env.variables.ddEnv>
    tags.datadoghq.com/service: orders-api
    tags.datadoghq.com/version: <+artifact.tag>
spec:
  template:
    metadata:
      labels:
        tags.datadoghq.com/env: <+env.variables.ddEnv>
        tags.datadoghq.com/service: orders-api
        tags.datadoghq.com/version: <+artifact.tag>
```

With a `version` tag set from `<+artifact.tag>`, you can scope queries to the new version, and distinguish canary data from baseline by `version` as well as by Pod. This also gives you the best experience on Datadog's own deployment tracking features.

10.10 Cost, load, and API usage

Each Verify step issues queries to Datadog every analysis interval, for each metric and log definition. A pipeline that fans out to 20 services with 15 queries each multiplies the API calls. Datadog applies rate limits per organization and per endpoint. If you see throttling:

- Reduce metric count to the signals that matter (the four golden signals per service).
- Share health sources through templates rather than duplicating similar queries.
- Stagger parallel verifications with lower `maxConcurrency` in multi-service stages.
- Ask your Datadog administrator about rate limit increases for the integration key.

10.11 Putting Datadog and rollback together

The following stage does a canary deployment, verifies with Datadog, rolls back automatically on a verification failure, and only then promotes:

```
- stage:
    name: deploy_prod_eu
    identifier: deploy_prod_eu
    type: Deployment
    spec:
      deploymentType: Kubernetes
      service:
        serviceRef: orders_api
        serviceInputs:
          serviceDefinition:
            type: Kubernetes
            spec:
              artifacts:
                primary:
                  primaryArtifactRef: <+input>
                  sources: <+input>
      environment:
        environmentRef: prod_eu
        deployToAll: false
        infrastructureDefinitions:
          - identifier: prod_eu_cluster
```

```
    execution:
      steps:
        - stepGroup:
            name: Canary
            identifier: canary
            steps:
              - step:
                  type: K8sCanaryDeploy
                  name: Canary Deploy
                  identifier: canary_deploy
                  timeout: 10m
                  spec:
                    instanceSelection:
                      type: Percentage
                      spec:
                        percentage: 10
                    skipDryRun: false
              - step:
                  type: Verify
                  name: Verify Canary
                  identifier: verify_canary
                  timeout: 1h
                  spec:
                    type: Canary
                    monitoredService:
                      type: Default
                      spec: {}
                    spec:
                      sensitivity: HIGH
                      duration: 15m
                      deploymentTag: <+serviceConfig.artifacts.primary.tag>
                      failOnNoAnalysis: true
                  failureStrategies:
                    - onFailure:
                        errors:
                          - Verification
                        action:
                          type: StageRollback
              - step:
                  type: K8sCanaryDelete
                  name: Canary Delete
                  identifier: canary_delete
                  timeout: 10m
                  spec: {}
        - step:
            type: K8sRollingDeploy
            name: Promote Rolling
            identifier: promote_rolling
            timeout: 15m
            spec:
              skipDryRun: false
              pruningEnabled: false
      rollbackSteps:
        - step:
            type: K8sCanaryDelete
            name: Rollback Canary Delete
            identifier: rollback_canary_delete
            timeout: 10m
            spec: {}
        - step:
            type: K8sRollingRollback
            name: Rollback Rolling
            identifier: rollback_rolling
            timeout: 10m
            spec:
              pruningEnabled: false
    failureStrategies:
      - onFailure:
          errors:
```

```
        - AllErrors
      action:
        type: StageRollback
```

If the Verify step fails, the `Verification` failure strategy triggers `StageRollback`, which runs `rollbackSteps` in order: delete the canary, then restore the previous rolling release. The stable version never changed. Customers saw at most ten percent of traffic touch the canary for the verification window.

10.12 Verification design checklist

1. Tag workloads with Unified Service Tagging including `version`.
2. Write queries grouped by Pod, scoped by service and environment.
3. Start with a small, high-signal set: error rate, p95 latency, saturation, and a log query for errors.
4. Add fail-fast thresholds for hard limits.
5. Use canary verification where possible; size the canary for statistical power.
6. Use `failOnNoAnalysis: true` in production once data is reliable.
7. Run in `ManualIntervention` mode until the signal is trusted.
8. Test the failure path with a deliberately bad build in a non-production environment and confirm the rollback works.

11. Rollbacks in Kubernetes

11.1 What rollback means in Harness

"Rollback" covers several different things, and mixing them up causes bad designs.

Kind	Trigger	What it does
Stage rollback	Failure strategy `StageRollback` during a run.	Runs the stage's `rollbackSteps` to restore the previous state of that stage's deployment.
Pipeline rollback	Failure strategy `PipelineRollback` or a manual action.	Rolls back the stages that already completed in the run, in reverse order, then the failing one.
Step rollback	A step-group-level rollback section.	Restores the work of a step group.
Post-deployment rollback	A user action after a successful deployment.	Reverts a running service to a prior successful deployment.
Native Kubernetes rollback	`kubectl rollout undo`.	Kubernetes-only; Harness is unaware of it unless you wrap it in a step.

The first two happen during a run. The fourth is a separate operational action. We cover each in turn.

11.2 How Harness tracks releases

Rollback needs a record of what the cluster looked like before. For Kubernetes, Harness maintains a **release history** in the cluster, keyed by the infrastructure's release name. Each successful or failed deployment appends a release entry listing the resources applied, with their versions, and the release status.

When a rolling rollback runs, Harness:

1. Reads the release history for the release name.
2. Finds the **last successful release** before the current one.
3. Re-applies the manifests recorded for that release, including the versioned ConfigMaps and Secrets it used.
4. Deletes resources that exist only in the failed release, as needed.
5. Waits for steady state.
6. Marks the release entries accordingly.

> **Where history lives**
>
> Harness stores the release record in the target cluster, in the namespace of the release (as a ConfigMap or Secret depending on the delegate and settings). RBAC for the delegate's service account must allow reading and writing these objects. If the delegate loses these permissions, rollbacks fail even though deployments appear to work.

11.2.1 What "last successful release" means for a first deployment

The first deployment of a service to a namespace has no earlier release. If it fails, rollback has nothing to return to. Harness handles this by removing the resources created by the failed first release (for rolling). Your service will be absent, not running an old version. That is correct, but it surprises teams who expect rollback to always leave something running. Pre-create the workload or accept the empty outcome.

11.2.2 Versioned ConfigMaps and Secrets

By default, Harness appends a version suffix to ConfigMaps and Secrets in the manifest (for example `orders-config-7`) and rewrites the references in workloads to match. Each deployment creates a new numbered copy. A rollback restores the earlier Deployment spec which points at the earlier numbered ConfigMap, so **configuration returns to the previous state together with the image**.

If you opt out, using `harness.io/skip-versioning: "true"` on the ConfigMap, a rollback restores the Deployment but the ConfigMap content stays at the **new** values, producing a mismatched, partially rolled back system. That is a major class of "rollback worked but the service is still broken" incidents.

Setting	Effect on rollback
Versioning enabled (default)	Config and workload roll back together.
`skip-versioning` on a resource	That resource is not restored by rollback.
`skipResourceVersioning: true` in manifest spec	Disables versioning for all ConfigMaps and Secrets. Use only when you manage config rollback another way.

11.3 Rolling rollback

The standard pairing is `K8sRollingDeploy` in `steps` and `K8sRollingRollback` in `rollbackSteps`:

```
execution:
  steps:
    - step:
        type: K8sRollingDeploy
        name: Rollout
        identifier: rollout
        timeout: 10m
        spec:
          skipDryRun: false
          pruningEnabled: false
  rollbackSteps:
    - step:
        type: K8sRollingRollback
        name: Rollback
        identifier: rollback
        timeout: 10m
        spec:
          pruningEnabled: false
failureStrategies:
  - onFailure:
      errors:
        - AllErrors
      action:
        type: StageRollback
```

11.3.1 What the rollback step does

Given a failed rollout, `K8sRollingRollback`:

- Reverts the managed workload to the previous revision's recorded spec.
- Reverts versioned ConfigMaps and Secrets by pointing the workload at the previous numbered copies.
- Removes resources that were newly created in the failed release.
- Waits for the rollback to reach steady state.

11.3.2 Pruning

With `pruningEnabled: true` on **both** the rollout and the rollback step:

- On rollout, Harness deletes resources that were part of the previous release but are not in the new manifests.
- On rollback, it deletes resources that were created by the failed release and restores those that were pruned.

Pruning keeps clusters tidy but is a destructive operation, and mistakes (for example, a manifest path change that omits a resource) can delete live resources. Use it deliberately, in environments where manifests are the single source of truth. Add `harness.io/skip-pruning: "true"` on resources that must never be pruned, such as PersistentVolumeClaims.

11.3.3 When rolling rollback does not help

Situation	Why
Failure before the apply (for example, manifest render error)	Nothing changed. Rollback step is skipped or a no-op.
Failed dry run	No resources were applied.

Situation	Why
Data migrations already applied	Rollback restores code, not data.
Custom resources not tracked as managed workloads	Harness restores what it tracked. Controllers' side effects remain.
Jobs and CronJobs	Past Job runs are not "undone".

11.4 Canary rollback

For canary, rollback is about removing the canary and leaving the stable workload alone.

```
rollbackSteps:
  - step:
      type: K8sCanaryDelete
      name: Canary Delete
      identifier: rollback_canary_delete
      timeout: 10m
      spec: {}
  - step:
      type: K8sRollingRollback
      name: Rolling Rollback
      identifier: rollback_rolling
      timeout: 10m
      spec:
        pruningEnabled: false
```

Order matters: delete the canary first so it stops taking traffic. Then restore the primary if it had been touched. When the failure occurs **before** the promote step, the primary workload has not changed, and the rolling rollback has little to do. When the failure occurs during the promote step (the full rollout), the rolling rollback does the real work.

11.4.1 Canary failure timeline

```
  t0  K8sCanaryDeploy  -> canary Pods running (10% traffic)
  t1  Verify           -> FAIL (error rate high)
  t2  StageRollback    -> K8sCanaryDelete removes canary
  t3  K8sRollingRollback -> primary already at old version, steady state confirmed
  Result: users impacted only between t0 and t2 for a fraction of traffic.
```

11.5 Blue-green rollback

Blue-green rollback is a swap back:

```
rollbackSteps:
  - step:
      type: K8sBGSwapServices
      name: Swap Back
      identifier: swap_back
      timeout: 10m
      spec: {}
```

The swap exchanges the selectors of the primary and stage Services. If the swap has already happened and the new version proves bad, swapping again points the primary Service back at the old workload, which is still running (unless scaled down). That makes blue-green rollback the fastest of the strategies, typically seconds.

11.5.1 Scale-down and the rollback window

After a successful swap, Harness can scale down the old workload to save cost. Once scaled down, the instant-rollback advantage disappears, since rolling back requires scaling the old version back up. Decide your rollback window: keep the old version running for 30 minutes or an hour after the swap, and scale down in a later step.

```
- step:
    type: K8sBlueGreenStageScaleDown
```

```
    name: Scale Down Old
    identifier: scale_down_old
    timeout: 10m
    spec: {}
```

Place it after verification and after the agreed observation period, for example after a `Wait` step or a later approval.

11.6 Failure strategies that trigger rollback

Rollback does not happen unless a failure strategy asks for it. In Harness:

```
failureStrategies:
  - onFailure:
      errors:
        - AllErrors
      action:
        type: StageRollback
```

Action	Effect
`StageRollback`	Runs `rollbackSteps` of the current stage.
`PipelineRollback`	Runs rollback for completed stages in reverse order, plus the failing stage.
`StepGroupRollback`	Runs the rollback section of the step group.
`ManualIntervention`	Pauses for a human decision.
`Retry`	Retries the failing step.
`Abort`	Stops the pipeline with no rollback.
`MarkAsSuccess`	Treats the failure as success.
`Ignore`	Continues.

If a stage has **no** failure strategy for an error type, the pipeline fails with no rollback and leaves the cluster in whatever state it reached. That is a frequent surprise: `rollbackSteps` are defined but never run, because nothing triggers them. Always confirm the failure strategy.

> **Rollback steps are not automatic**
>
> Defining `rollbackSteps` does nothing by itself. Only a failure strategy with a rollback action (or a manual rollback request) executes them. When you copy a stage between pipelines, verify the failure strategy came along.

11.6.1 Which errors to match

Error type	Examples
`AllErrors`	Everything. The simplest catch-all.
`Timeout`	Step or stage timeouts.
`Verification`	Verify step failures.
`Authentication`, `Authorization`, `Connectivity`	Connector and permission problems.
`DelegateProvisioning`, `DelegateRestart`	Delegate problems.
`PolicyEvaluationFailure`	OPA policy failures.
`UserMarkedFailure`	A person marked the step failed (for example a rejected approval).

For infrastructure problems like `Connectivity`, rollback may not be appropriate because the cluster is unreachable. Use `Retry` or `ManualIntervention` for those, and rollback for application-level errors.

```
failureStrategies:
  - onFailure:
      errors:
        - Connectivity
```

```
        - DelegateProvisioning
      action:
        type: Retry
        spec:
          retryCount: 3
          retryIntervals:
            - 30s
            - 1m
            - 2m
          onRetryFailure:
            action:
              type: ManualIntervention
              spec:
                timeout: 1h
                onTimeout:
                  action:
                    type: Abort
  - onFailure:
      errors:
        - AllErrors
      action:
        type: StageRollback
```

Failure strategies are evaluated in order. The first one whose error list matches the failure applies. List specific errors first and `AllErrors` last.

11.7 Pipeline rollback

When several stages deploy parts of a release, a failure in the last stage may call for rolling back everything that already completed.

```
failureStrategies:
  - onFailure:
      errors:
        - AllErrors
      action:
        type: PipelineRollback
```

Execution order on rollback: the failing stage first, then the previously completed stages in reverse order. Each stage executes its own `rollbackSteps`. A pipeline that deploys the database proxy, then the API, then the web tier, rolls back in the order web, API, proxy.

Pipeline rollback works well when:

- Every deployment stage has correct `rollbackSteps`.
- Stages are idempotent and independent of manual side effects.
- You want atomic release semantics.

It works badly when some stages have no rollback steps (for example, a database migration) because the rollback proceeds past them silently. Document stages that cannot be rolled back and arrange compensating actions.

11.8 Rolling back after a successful deployment

Sometimes the problem appears hours later, when the pipeline is long finished. Options:

1. **Re-run the pipeline with the previous artifact tag.** Simple and auditable. It performs a normal forward deployment of the older version through the whole process, including approvals, which is slow in an emergency.
2. **An emergency rollback pipeline.** A small pipeline that deploys a chosen older version with abbreviated gates, an emergency change record, and verification. Prepare it before you need it.
3. **Post-deployment rollback.** Harness can revert a service in an environment to its previous successful deployment through a rollback action in the UI or API, for supported deployment types, using the stored rollback artifact and the recorded release information.

Availability

Post-deployment rollback is gated by an account setting and supports specific deployment types. Confirm in your Harness account that it is enabled, and rehearse it on a non-production service. It reverts the workload to the previously recorded state; it does not revert databases or external systems.

11.8.1 Rollback pipeline pattern

```
pipeline:
  name: orders-emergency-rollback
  identifier: orders_emergency_rollback
  projectIdentifier: payments
  orgIdentifier: platform
  variables:
    - name: targetTag
      type: String
      value: <+input>
    - name: reason
      type: String
      value: <+input>
    - name: environment
      type: String
      value: <+input>.allowedValues(staging,prod_eu,prod_us)
  stages:
    - stage:
        name: rollback_deploy
        identifier: rollback_deploy
        type: Deployment
        spec:
          deploymentType: Kubernetes
          service:
            serviceRef: orders_api
            serviceInputs:
              serviceDefinition:
                type: Kubernetes
                spec:
                  artifacts:
                    primary:
                      primaryArtifactRef: main_image
                      sources:
                        - identifier: main_image
                          type: DockerRegistry
                          spec:
                            tag: <+pipeline.variables.targetTag>
          environment:
            environmentRef: <+pipeline.variables.environment>
            deployToAll: false
            infrastructureDefinitions: <+input>
          execution:
            steps:
              - step:
                  type: K8sRollingDeploy
                  name: Roll Back To Tag
                  identifier: roll_back_to_tag
                  timeout: 10m
                  spec:
                    skipDryRun: true
                    pruningEnabled: false
              - step:
                  type: Verify
                  name: Verify
                  identifier: verify
                  timeout: 30m
                  spec:
                    type: Rolling
                    monitoredService:
                      type: Default
```

```
                  spec: {}
                spec:
                  sensitivity: MEDIUM
                  duration: 5m
                  failOnNoAnalysis: false
        rollbackSteps: []
      failureStrategies:
        - onFailure:
            errors:
              - AllErrors
            action:
              type: ManualIntervention
              spec:
                timeout: 30m
                onTimeout:
                  action:
                    type: Abort
```

This is a forward deploy of an older tag, so there is no "rollback" in the Harness sense. It is the operationally simplest route and works for any deployment type. Because it is a forward deploy, rolling back behavior of its own failures is a human decision: manual intervention.

11.9 Rollback hazards specific to Kubernetes

11.9.1 StatefulSets and PersistentVolumes

StatefulSets roll back in reverse ordinal order and may need manual intervention when a Pod is stuck. PersistentVolumeClaims are not reverted. Data written by the new version stays. Test restore procedures separately.

11.9.2 CustomResourceDefinitions

CRDs are cluster-scoped and shared. Rolling back a CRD change can orphan or invalidate existing custom resources. Deploy CRDs through a separate, carefully reviewed pipeline, and prefer additive changes.

11.9.3 Jobs and migrations

A Job created by the failed release is removed on rollback if it was tracked. A migration that already ran has changed your data. Plan forward-fix migrations and keep schema changes backward compatible for at least one release.

11.9.4 HPA and replica counts

When a HorizontalPodAutoscaler manages replicas, avoid setting `replicas` in the Deployment manifest; otherwise every deploy resets the autoscaler's decision. After rollback, the replica count reverts to the manifest value if you set one.

11.9.5 Immutable fields

Some Kubernetes fields are immutable (for example, Service `clusterIP`, or a Deployment's selector). A change to an immutable field makes apply fail. Rollback may also fail if the previous manifest conflicts with the current live object. Delete and recreate is sometimes required, which Harness will not do for you unless the step is configured for it.

11.9.6 Image availability

Rollback re-deploys an old image. If your registry retention policy has deleted it, the rollback fails with `ImagePullBackOff`. Set registry retention to keep at least the last several production images, and consider digest pinning.

11.9.7 Steady state of the rollback itself

A rollback must reach steady state too. If the cluster is under resource pressure (the reason for the failure), the rollback can stall. Set the rollback step timeout generously compared to the rollout timeout, or consider a pre-rollback step that scales down non-critical workloads.

11.10 Native `kubectl rollout undo` versus Harness rollback

Aspect	`kubectl rollout undo`	Harness rollback
Scope	One Deployment's ReplicaSet history.	The whole release: workload, ConfigMaps, Secrets, created resources.
Config restored	No (ConfigMaps are not versioned by Kubernetes).	Yes, with versioning.
Audit	Not in Harness.	Recorded in the execution.
Works when	The Deployment exists with revision history.	The release history exists in the cluster.

Prefer Harness rollback in Harness-managed workloads. Use `kubectl rollout undo` only as an emergency tool, and then reconcile the Harness release by redeploying a known good version afterward, so Harness's record matches reality.

11.11 Testing rollbacks

Untested rollbacks are hope, not a design.

1. **Deliberate failure test.** Deploy an image that fails its readiness probe to a non-production environment. Confirm the rollout times out, the failure strategy fires, and the previous version is running again.
2. **Config drift test.** Change a ConfigMap value and the image together, fail the rollout, and confirm the ConfigMap returns to the old value.
3. **Verification failure test.** Deploy a build that raises error rates and confirm Verify fails and the canary is removed.
4. **Rollback of the rollback.** Redeploy the new version after a rollback and confirm release history is intact.
5. **Permission test.** Confirm the delegate's service account can read and write the release history objects in a fresh namespace.

Record the observed time to recover. That number belongs in your runbook.

11.12 Rollback design checklist

Check	Done
Every deployment stage has `rollbackSteps` matching its strategy.	
Every deployment stage has a failure strategy that triggers rollback.	
`Connectivity` and delegate errors use retry, not rollback.	
ConfigMaps and Secrets are versioned.	
Image retention keeps recent production images.	
Schema changes are backward compatible.	
Blue-green scale-down waits for the rollback window.	
An emergency rollback pipeline exists and has been rehearsed.	
Notifications fire on rollback events.	
ServiceNow tickets are updated on rollback.	

12. Failure Strategies, Conditional Execution, and Step Groups

12.1 Failure strategies in depth

A failure strategy answers: **when this node fails with this kind of error, what should the pipeline do?** Failure strategies can be set at the pipeline, stage, step group, and step levels. The most specific level that matches wins.

12.1.1 Structure

```
failureStrategies:
  - onFailure:
      errors:
        - Timeout
      action:
        type: Retry
        spec:
          retryCount: 2
          retryIntervals:
            - 1m
            - 5m
          onRetryFailure:
            action:
              type: StageRollback
  - onFailure:
      errors:
        - AllErrors
      action:
        type: Abort
```

Each entry has a list of error types and an action. The runtime checks entries in order and applies the first match.

12.1.2 Action reference

Action	Parameters	Notes
`StageRollback`	none	Execute the stage's rollback steps.
`PipelineRollback`	none	Roll back completed stages and the failing one.
`StepGroupRollback`	none	Execute the step group's rollback steps.
`Retry`	`retryCount`, `retryIntervals`, `onRetryFailure`	Retry the failed node. After retries are exhausted, run `onRetryFailure`.
`ManualIntervention`	`timeout`, `onTimeout`	Pause for human choice.
`MarkAsSuccess`	none	Treat as success.
`MarkAsFailure`	none	Used to convert behaviors; marks as failed.
`Ignore`	none	Continue past the failure.
`Abort`	none	Terminate the execution.

12.1.3 Error types

Error type	Meaning
`AllErrors`	Matches any failure.
`Timeout`	Step exceeded its timeout.
`Verification`	Continuous verification failed.
`Authentication`	Credentials rejected.

Error type	Meaning
`Authorization`	Permission denied.
`Connectivity`	Network failure reaching a system.
`DelegateProvisioning`	No delegate available for the task.
`DelegateRestart`	Delegate restarted during the task.
`PolicyEvaluationFailure`	An OPA policy failed.
`Unknown`	Unclassified failures.
`UserMarkedFailure`	A person marked the step failed.

12.1.4 Manual intervention in depth

Manual intervention pauses the node and presents choices to authorized users:

Choice	Effect
Mark as Success	Continue as if the step passed.
Ignore Failure	Continue; the failure remains recorded.
Retry	Run the step again.
Rollback Stage / Pipeline	Run rollback steps.
Abort	End the execution.
Mark as Failure	Fail definitively.

Combine with `onTimeout` so unattended pauses resolve to a safe default.

```
failureStrategies:
  - onFailure:
      errors:
        - Verification
      action:
        type: ManualIntervention
        spec:
          timeout: 20m
          onTimeout:
            action:
              type: StageRollback
```

> **Who can respond**
>
> Responding to a manual intervention requires execution permissions. If nobody on-call holds them, the pause will always run to timeout. Check RBAC for the on-call group.

12.2 Conditional execution

Conditional execution decides whether a stage or step runs at all, using the **status of previous nodes** and optional **JEXL conditions**.

```
when:
  stageStatus: Success
  condition: <+ <+env.type> == "Production" >
```

12.2.1 Status options

`stageStatus` or `pipelineStatus`	Runs when
`Success`	Previous stages (or the pipeline so far) succeeded. Default.
`Failure`	A previous stage failed. Good for cleanup and notification stages.

`stageStatus` or `pipelineStatus`	Runs when
`All`	Always, regardless of status.

For steps: `when: stageStatus: Success | Failure | All`, with an optional `condition`.

12.2.2 Typical uses

Use	Expression
Run only for production	`<+ <+env.type> == "Production" >`
Skip verification in dev	`<+ <+env.name> != "dev" >`
Run only when a flag input is set	`<+ <+pipeline.variables.runSmokeTests> == "true" >`
Run a stage only on weekday business hours	Compute with a Shell Script step, output a variable, then use that variable in the condition.
Notification on failure	`stageStatus: Failure` or `pipelineStatus: Failure`
Skip approval for standard changes	`<+ <+pipeline.variables.changeType> != "standard" >`

12.2.3 Cleanup stages

A stage with `when: pipelineStatus: All` runs even after earlier failure. Use it for always-do work such as closing tickets or deleting preview namespaces.

```
- stage:
    name: cleanup
    identifier: cleanup
    type: Custom
    when:
      pipelineStatus: All
    spec:
      execution:
        steps:
          - step:
              type: ShellScript
              name: Delete Preview Namespace
              identifier: delete_preview
              timeout: 5m
              spec:
                shell: Bash
                onDelegate: true
                source:
                  type: Inline
                  spec:
                    script: kubectl delete namespace orders-<+pipeline.variables.previewName> --ignore-not-found
                environmentVariables: []
                outputVariables: []
```

`Custom` stages are lightweight stages for steps that need no service or environment.

12.3 Step groups

A step group bundles steps for shared failure strategy, rollback, conditional execution, and looping.

```
- stepGroup:
    name: Smoke Tests
    identifier: smoke_tests
    steps:
      - step:
          type: Http
          name: Health
          identifier: health
          timeout: 30s
          spec:
```

```
          url: https://orders.example.com/health
          method: GET
          headers: []
          assertion: <+httpResponseCode> == 200
    - step:
        type: Http
        name: Version
        identifier: version
        timeout: 30s
        spec:
          url: https://orders.example.com/version
          method: GET
          headers: []
          assertion: <+httpResponseBody>.contains("<+artifact.tag>")
  failureStrategies:
    - onFailure:
        errors:
          - AllErrors
        action:
          type: StepGroupRollback
  rollbackSteps:
    - step:
        type: ShellScript
        name: Flag Off
        identifier: flag_off
        timeout: 2m
        spec:
          shell: Bash
          onDelegate: true
          source:
            type: Inline
            spec:
              script: ./disable-feature-flag.sh
          environmentVariables: []
          outputVariables: []
```

12.3.1 Why step groups help

- **Scoped rollback**: undo only what the group did.
- **Shared strategy**: one failure policy for related steps.
- **Looping**: a group can be repeated per tenant, per region.
- **Readability**: execution graphs collapse a group into one node.

12.4 Timeouts at every level

Level	Setting	Behavior
Step	`timeout`	Fails the step with a `Timeout` error.
Step group	none directly	Governed by member steps.
Stage	none by default	Governed by steps; approvals have their own timeout.
Pipeline	Optional timeout in pipeline settings	Terminates the execution.
Approval	`timeout` on the step	Long durations such as `1d` are normal.
Delegate task	Platform-level limits	Long tasks may be bounded by delegate settings.

Set explicit timeouts everywhere. A missing timeout is rarely a problem during testing and often a problem at 2 a.m.

12.5 Retries and idempotency

Retries only help if repeating the step is safe. Check each step for idempotency:

Step	Idempotent?	Notes
`K8sRollingDeploy`	Generally yes.	Reapplies the same manifests.
`K8sApply` of Jobs	Not always.	A Job name collision fails on retry. Use generated names.
`ServiceNowCreate`	No.	A retry creates a second ticket. Avoid retry, or check for an existing ticket first.
`Http` POST	Depends on API.	Use idempotency keys.
`ShellScript`	Depends on script.	Write scripts to tolerate re-runs.

A retry strategy on `ServiceNowCreate` is a classic trap that floods the change queue.

12.6 Execution input and pauses

Besides approvals, you can pause for human data with `<+input>.executionInput()` on fields, or add a Harness Approval with `approverInputs` to collect values at the gate. Pick approvals when you want audit and authorization, and execution inputs when the person running the pipeline can decide.

12.7 Notifications

Notification rules attach to pipelines (and stages and steps for some events) and deliver to Slack, email, PagerDuty, Microsoft Teams, or webhooks.

Event	Typical recipient
Pipeline start	Release channel
Stage success or failure	Service team
Approval pending	Approver group
Approval expiry	Release manager
Pipeline failed	On-call
Step failed (verification)	Service owner

Use message templates including `<+pipeline.executionUrl>` and `<+pipeline.triggeredBy.name>` so recipients can click straight to the run.

12.8 Design patterns

12.8.1 The safe production stage

```
[Approval stage]  ServiceNow change approved, within window
       |
[Deploy stage]    cluster guard -> canary -> Verify -> promote
       |            failure: Verification -> StageRollback
       |            failure: Connectivity -> Retry then ManualIntervention
       v
[Post stage]      update ticket, notify (when: pipelineStatus All)
```

12.8.2 The fan-out with a single approval

```
[Approval]  ->  [Deploy services x regions (fan-out)]  ->  [Summary]
```

12.8.3 The guarded promotion

```
dev -> (auto)  -> qa -> (verify) -> staging -> (approval + verify) -> prod
```

Service propagation keeps the artifact identical across all stages.

13. Governance: Templates, Policy, Freeze Windows, RBAC, and Triggers

13.1 Why governance belongs in the pipeline

At small scale, a good pipeline and careful engineers are enough. At the scale of many teams, controls must be **enforced by the platform**, not remembered by people. Harness provides several levers:

Lever	Enforces
Templates	Consistent structure and steps across teams.
OPA policies	Rules on pipeline definitions and executions.
Freeze windows	No deployments during defined periods.
RBAC and resource groups	Who can do what to which entities.
Approvals	Human or system sign-off.
Audit trail	Evidence of who did what.
Secrets management	Credentials never in YAML.

13.2 Templates

A template is a reusable definition of a **step**, **step group**, **stage**, or **pipeline**, versioned with a version label, and scoped to account, org, or project.

13.2.1 Stage template with runtime inputs

```yaml
template:
  name: k8s_canary_verified_stage
  identifier: k8s_canary_verified_stage
  versionLabel: v1
  type: Stage
  projectIdentifier: payments
  orgIdentifier: platform
  spec:
    type: Deployment
    spec:
      deploymentType: Kubernetes
      service:
        serviceRef: <+input>
        serviceInputs: <+input>
      environment:
        environmentRef: <+input>
        deployToAll: false
        infrastructureDefinitions: <+input>
      execution:
        steps:
          - stepGroup:
              name: Canary
              identifier: canary
              steps:
                - step:
                    type: K8sCanaryDeploy
                    name: Canary Deploy
                    identifier: canary_deploy
                    timeout: 10m
                    spec:
                      instanceSelection:
                        type: Percentage
                        spec:
```

```
                    percentage: <+input>.default(10)
                skipDryRun: false
            - step:
                type: Verify
                name: Verify Canary
                identifier: verify_canary
                timeout: 1h
                spec:
                  type: Canary
                  monitoredService:
                    type: Default
                    spec: {}
                  spec:
                    sensitivity: <+input>.default(MEDIUM).allowedValues(LOW,MEDIUM,HIGH)
                    duration: <+input>.default(10m)
                    deploymentTag: <+serviceConfig.artifacts.primary.tag>
                    failOnNoAnalysis: true
            - step:
                type: K8sCanaryDelete
                name: Canary Delete
                identifier: canary_delete
                timeout: 10m
                spec: {}
        - step:
            type: K8sRollingDeploy
            name: Promote
            identifier: promote
            timeout: 15m
            spec:
              skipDryRun: false
              pruningEnabled: false
      rollbackSteps:
        - step:
            type: K8sCanaryDelete
            name: Rollback Canary Delete
            identifier: rollback_canary_delete
            timeout: 10m
            spec: {}
        - step:
            type: K8sRollingRollback
            name: Rollback Rolling
            identifier: rollback_rolling
            timeout: 15m
            spec:
              pruningEnabled: false
  failureStrategies:
    - onFailure:
        errors:
          - AllErrors
        action:
          type: StageRollback
```

Teams consume the template in a pipeline:

```
- stage:
    name: deploy_prod_eu
    identifier: deploy_prod_eu
    template:
      templateRef: k8s_canary_verified_stage
      versionLabel: v1
      templateInputs:
        type: Deployment
        spec:
          service:
            serviceRef: orders_api
          environment:
            environmentRef: prod_eu
```

```
            infrastructureDefinitions:
              - identifier: prod_eu_cluster
```

13.2.2 Versioning strategy

- Use **semantic version labels** (`v1`, `v1.1`, `v2`).
- Mark a **stable** version and allow pipelines to follow it (`__STABLE__`) for rapid, controlled propagation of fixes, or pin to exact versions for conservative teams.
- Treat a change to a stage template as a platform release: review, test with a sandbox pipeline, then publish a new version.
- Use **template usage** views to find which pipelines reference a template before changing it.

13.2.3 Who can edit templates

Restrict template edit rights to a platform team through RBAC. Allow consumption by everyone. Account-level templates reach all organizations, so grant edit rights narrowly.

13.3 OPA policies

Harness embeds the Open Policy Agent. Policies are written in Rego and attached to **policy sets**, which are bound to **entity types** and **events**. Evaluation occurs on events such as pipeline save and pipeline run.

13.3.1 Example: require approval for production deployments

```
package pipeline_environment

deny[msg] {
  stage := input.pipeline.stages[_].stage
  stage.type == "Deployment"
  stage.spec.environment.environmentRef == "prod_eu"
  not has_approval_before(stage)
  msg := sprintf("Stage '%s' deploys to production without a preceding approval stage", [stage.name])
}

has_approval_before(stage) {
  prior := input.pipeline.stages[_].stage
  prior.type == "Approval"
}
```

This is a simplified illustration. A production policy should verify ordering and the type of approval. Use the policy editor's testing panel with sample pipeline JSON before enforcing.

13.3.2 Example: forbid skipping dry run in production

```
package pipeline_k8s

deny[msg] {
  stage := input.pipeline.stages[_].stage
  step := stage.spec.execution.steps[_].step
  step.type == "K8sRollingDeploy"
  step.spec.skipDryRun == true
  msg := sprintf("Step '%s' must not skip dry run", [step.name])
}
```

13.3.3 Example: require verification in production stages

```
package pipeline_verify

has_verify(stage) {
  step := stage.spec.execution.steps[_].step
  step.type == "Verify"
}
```

```
has_verify(stage) {
  sg := stage.spec.execution.steps[_].stepGroup
  step := sg.steps[_].step
  step.type == "Verify"
}

deny[msg] {
  stage := input.pipeline.stages[_].stage
  stage.type == "Deployment"
  startswith(stage.spec.environment.environmentRef, "prod")
  not has_verify(stage)
  msg := sprintf("Production stage '%s' has no Verify step", [stage.name])
}
```

Policy sets can be set to **warn** (non-blocking) or **error** (blocking). Introduce policies in warn mode to measure impact, then switch to enforce.

> **Policies on templates**
>
> Enforcing through templates (the structure is correct by construction) is friendlier than policies that reject pipelines after the fact. Use policies as a backstop for teams that bypass templates.

13.4 Freeze windows

A freeze window blocks deployments (and optionally other actions) during defined periods, such as holiday seasons or end-of-quarter.

Attribute	Options
Scope	Account, org, or project.
Target	All environments, only Production, specific services or environments, or specific pipelines.
Schedule	One-time or recurring, with time zone.
Notification	Before the window starts and when it begins or ends.
Exemptions	Specific services or environments excluded from the freeze.

When a freeze is active, a run that targets a frozen entity is blocked, with a message to the user. Users with the right permission can override a freeze in emergencies, depending on RBAC.

```
freeze:
  name: Year End Freeze
  identifier: year_end_freeze
  entityConfigs:
    - name: prod_all
      entities:
        - type: Service
          filterType: All
        - type: Environment
          filterType: All
        - type: EnvType
          filterType: Production
  status: Enabled
  description: No production changes during year end.
  windows:
    - timeZone: Asia/Kolkata
      startTime: 2026-12-24 18:00 PM
      duration: 10d
```

The shape of freeze YAML changes; create one in the UI and copy the generated YAML for your repository.

13.4.1 Freezes and ServiceNow

A freeze window stops technical execution. A ServiceNow change window authorizes execution. They answer different questions, and mature processes use both. The freeze protects against **everything** during a global event, while the change window authorizes **one specific** change.

13.5 RBAC and resource groups

Harness RBAC assigns **roles** (sets of permissions) on **resource groups** (sets of entities) to **principals** (users, user groups, service accounts).

Role example	Permissions
Pipeline Developer	Create and edit pipelines, run non-production.
Release Manager	Execute pipelines in production, approve deployments.
Platform Admin	Edit templates, connectors, delegates.
Auditor	View executions and audit trail.

Principles:

1. **Least privilege.** Developers run dev and staging; release managers run production.
2. **Separate** who edits pipelines from who approves them.
3. Use **environment-scoped** permissions so production access is a different grant from dev.
4. Use **service accounts** with tokens for automation, not personal accounts.
5. Review access periodically.

13.6 Triggers

Triggers start pipelines automatically.

Trigger type	Starts when
Webhook (Git)	Push, pull request, tag, or release events.
Artifact	A new tag or version appears in a registry.
Manifest	A new Helm chart version appears.
Scheduled (cron)	At a time schedule.
Custom webhook	Any external system calls a URL.

13.6.1 Artifact trigger

```
trigger:
  name: orders_api_new_tag
  identifier: orders_api_new_tag
  enabled: true
  orgIdentifier: platform
  projectIdentifier: payments
  pipelineIdentifier: orders_release
  source:
    type: Artifact
    spec:
      type: DockerRegistry
      spec:
        connectorRef: registry_main
        imagePath: acme/orders-api
        tag: <+trigger.artifact.build>
        eventConditions:
          - key: <+trigger.artifact.build>
            operator: Regex
            value: ^1\.[0-9]+\.[0-9]+$
```

```
  inputSetRefs:
    - env_dev
```

The trigger runs the pipeline for each new matching tag. `<+trigger.artifact.build>` injects the tag. Pair with an input set for the remaining inputs.

13.6.2 Trigger safety

- Restrict automatic triggers to non-production. Let production promotion be a deliberate act or a gated downstream pipeline.
- Use event conditions (regex, branch filters) to avoid triggering on every tag.
- Use a service account identity for trigger-started runs so audit logs are meaningful.

13.7 Audit and compliance evidence

Each execution records inputs, resolved values, approvals, who approved and when, and logs. Retain them according to your policy. The audit trail records configuration changes. Together they can answer: who changed the pipeline, who ran it, who approved, what version went where.

Auditor question	Where to look
Who approved this production deploy?	Approval step output and audit trail.
Which change ticket authorized it?	ServiceNow step outputs and pipeline variables.
Which image was deployed?	Service step output (artifact, digest).
Was it verified?	Verify step results.
Who changed the pipeline last week?	Audit trail for the pipeline entity.
Was the freeze respected?	Freeze events in the execution.

13.8 Secrets

- Store secrets in an external secret manager (HashiCorp Vault, AWS Secrets Manager, GCP Secret Manager, Azure Key Vault) wherever possible, and reference them from Harness.
- Use **secret variable type** in services, not strings.
- Restrict secret scope. Project-level for service credentials, org or account only for shared infrastructure.
- Never print decoded secrets.
- Rotate on schedule; use short-lived credentials (OIDC, workload identity) for cloud connectors where supported.

13.9 Pipeline as code

Pipelines, templates, input sets, services, environments, and infrastructure definitions can be stored in Git (**remote entities**). Benefits: review, history, branching, and rollback of the pipeline definition itself.

Practices:

- Protect the main branch of the pipeline repository, and require review.
- Keep input sets next to pipelines.
- Use the same repository layout for every team (a `.harness/` folder with subfolders per entity type).
- Validate YAML in CI using the Harness API or the UI-supported validation, before merge.

14. Troubleshooting and Operations

14.1 A systematic approach

When a pipeline misbehaves, avoid guessing. Follow this order:

1. **Where did it fail?** Pipeline, stage, step, and which iteration if looped.
2. **What kind of failure?** Validation (before running), connectivity, timeout, step logic, verification, approval, or policy.
3. **What changed?** The pipeline YAML, template version, service, environment, override, artifact, cluster, delegate, or external system.
4. **What do the logs and inputs say?** Step logs, the Input tab (resolved values), and Output tab.
5. **Can I reproduce in a sandbox?** Use a Probe step and a non-production environment.

14.2 Failure classes and first checks

Class	Symptoms	First checks
Validation	Pipeline will not start; red errors in the form.	Required runtime inputs, regex constraints, input set mismatch, missing entity references.
Delegate	"No eligible delegate", tasks never start.	Delegate health, selectors, connector's delegate selectors, capacity.
Git or manifest	Fetch failures, template errors.	Connector credentials, branch or tag existence, path correctness, rendering errors in logs.
Kubernetes apply	Dry run fails, apply errors.	RBAC for the delegate service account, namespace exists, immutable fields, invalid manifest.
Steady state	Timeout waiting for rollout.	Pod events, readiness probe, image pull, resource quotas, node capacity.
Verification	No analysis or unexpected fail.	Queries in Datadog, tags, connector site, sensitivity.
Approval	Waiting forever or immediate fail.	Approver group, criteria, ticket fields, change window.
Rollback	Did not run, or failed.	Failure strategy, release history permissions, image retention.
Expression	Unresolved or wrong value.	Namespace spelling, timing, loop suffix.

14.3 Delegate troubleshooting

The delegate is the most common root cause of CD problems that are not application problems.

Symptom	Likely cause	Fix
Steps stay "queued"	No delegate matches selectors.	Check tags on delegates versus selectors in step, stage, and connector.
Intermittent connection failures	Delegate CPU or memory pressure.	Scale up or out. Check task concurrency.
Delegate offline	Network or egress changes, expired token.	Check logs, firewall, delegate token.
Works from a laptop, fails from delegate	Delegate network differs.	Test from the delegate pod with `curl` or `kubectl`.
Old behavior after upgrade	Delegate version drift.	Align delegate versions; enable auto upgrade or follow your upgrade runbook.
Task timeouts in large matrices	Too many concurrent tasks.	Lower `maxConcurrency`; add delegate replicas.

From the delegate pod you can test what the delegate sees:

```
kubectl exec -it deploy/harness-delegate -n harness-delegate-ng -- bash
curl -sS https://api.datadoghq.com/api/v1/validate -H "DD-API-KEY: ${DD_API_KEY:?set it first}"
```

```
curl -sS https://acme.service-now.com/api/now/table/change_request?sysparm_limit=1 -u "svc_harness:${SN_PASSWORD:?set it first}"
kubectl auth can-i create deployments -n orders
```

Replace the namespace and deployment names with yours. Never paste real credentials into shared terminals or logs.

14.4 Kubernetes-specific issues

14.4.1 Rollout stuck at steady state

```
kubectl -n orders get pods -l app=orders-api
kubectl -n orders describe pod <pod>
kubectl -n orders rollout status deployment/orders-api
kubectl -n orders get events --sort-by=.lastTimestamp | tail -30
```

Common root causes: failing readiness probes, `ImagePullBackOff` (wrong tag or registry credentials), insufficient resources (`Pending` Pods), admission webhook denials, `CrashLoopBackOff` from missing config.

14.4.2 Release history problems

Symptoms: rollback says there is no previous release; a rollback restores the wrong version.

Checks:

- Is the release name stable between runs? An environment that changed release name starts new history.
- Are two services sharing a release name in the same namespace?
- Does the delegate service account have rights to ConfigMaps or Secrets in the namespace?
- Was the release history manually deleted during cleanup?

14.4.3 Managed workload errors

"No managed workload found" or "multiple workloads found" indicates the manifests lack, or have more than one, qualifying workload. Annotate the workload to manage, or mark others as direct apply.

14.4.4 Immutable field errors

If a Service or Deployment selector changed, Kubernetes rejects the update. Delete the object (carefully) and redeploy, or avoid changing immutable fields.

14.5 Looping and multi-service problems

Symptom	Cause	Fix
Expression resolves for one iteration but not others	Identifier suffixes.	Use `<+strategy.identifierPostFix>` in references.
Barrier deadlocks in matrix	`maxConcurrency` below iteration count.	Remove or raise the limit.
One iteration failure stops all	Failure strategy at the parent level.	Adjust or use per-iteration strategy.
Inputs form is huge	Many services times inputs.	Use input sets; reduce input surface by using expressions.
Delegate overload	Too many parallel iterations.	Lower concurrency; add delegates.
Different regions behave differently	Overrides differ per environment.	Print `<+serviceVariables.*>` with a Probe step in each.

14.6 Variable and override problems

Symptom	Cause	Fix
Override ignored	Wrong scope, name mismatch, referencing `service.variables`.	Use `<+serviceVariables.name>`, check override scope and the exact variable name.
Value differs between runs	Runtime input or expression changed.	Compare the resolved Input tabs.
Secret prints as `*******` where you need it	By design.	Pass it through a secret-aware mechanism (environment variable from secret).
Input set marked invalid	Pipeline inputs changed.	Update the input set to the new schema.
"Unresolved expression"	Output not produced yet or typo.	Check timing and path.

14.7 ServiceNow troubleshooting

Symptom	Cause	Fix
401 or 403	Credentials or roles.	Test the connector. Verify the integration user's ACLs.
Ticket not found	Wrong number or table.	Confirm `ticketType` matches the ticket.
Criteria never met	Field representation.	Compare internal and display values. Try JEXL.
Change window failure	Window closed or fields mismatched.	Verify field names and time zones.
Duplicate tickets	Retry on a create step.	Remove retry from create steps.
Slow polling or throttling	Interval too short for many pipelines.	Increase `retryInterval`.

14.8 Datadog troubleshooting

See Chapter 10.9. Additionally:

Symptom	Cause	Fix
Connector test passes, but verification has no data	Key permission or query problem.	Run the query in Datadog and check the key's scope.
Verify always passes	Insensitive configuration or no data and `failOnNoAnalysis: false`.	Increase sensitivity; set `failOnNoAnalysis: true`.
Verify always fails	Metrics naturally noisy, baseline too short.	Use ignore thresholds for noisy metrics; extend duration.

14.9 Operating pipelines in production

14.9.1 Observability for the pipelines themselves

Treat the delivery system as a production service:

- Track **deployment frequency**, **lead time**, **change failure rate**, and **time to restore** for each service, for example from the Harness dashboards or through your own telemetry.
- Alert on a **pipeline failure rate** above a threshold.
- Monitor **delegate health** (CPU, memory, task backlog).
- Watch for **approval queues** that grow long.

14.9.2 Capacity planning

Resource	Consideration
Delegates	Peak parallel tasks times average task duration. Keep headroom.
Cluster API servers	Many simultaneous applies increase API load.

Resource	Consideration
Registries	Parallel image pulls on many nodes. Use caching or pre-pulls.
Datadog and ServiceNow APIs	Rate limits.

14.9.3 Disaster recovery

- Keep pipeline, service, environment, template definitions in Git.
- Keep delegate manifests in Git, rebuildable.
- Document the manual deployment path (a `kubectl` or Helm procedure) for the case where Harness is unavailable.
- Test restoring from nothing in a scratch project.

14.10 Incident playbook: bad release in production

1. **Stop the bleeding.** Abort running pipelines. Freeze further deployments if multiple services are at risk.
2. **Decide**: rollback or forward-fix. Use the emergency rollback pipeline or post-deployment rollback, and record a ServiceNow emergency change.
3. **Verify recovery** with the same Datadog signals as in deployment verification.
4. **Communicate** in the incident channel with the execution links.
5. **Preserve evidence**: execution URL, verification results, timeline.
6. **Retrospective**: why did verification or testing not catch it? Add a query, a threshold, or a test.

15. Capstone: A Production-Grade Release Pipeline

This chapter assembles every concept into one pipeline for the `orders` platform. It deploys three services through staging and two production regions, under change control, with verification, barriers, and rollback. Read it as an annotated design, not a copy-paste artifact. Replace identifiers with yours, and validate the YAML in your Pipeline Studio.

15.1 Requirements

#	Requirement	Feature used
R1	One artifact version per service flows unchanged through all stages.	Input sets, digest pinning
R2	Staging validates all three services together.	Multi-service stage, Verify
R3	Production requires an approved ServiceNow change inside its window.	`ServiceNowApproval`, change window
R4	Production regions deploy in parallel but cut over together.	Matrix (region), Barrier
R5	Bad canaries are rolled back automatically using Datadog signals.	`Verify`, `StageRollback`
R6	The change ticket always reflects the outcome.	`ServiceNowUpdate`, cleanup stage
R7	Per-environment tuning without pipeline edits.	Service variables, environment overrides
R8	Only one production release at a time.	Queue step, infrastructure queueing
R9	Mistyped change numbers fail immediately.	Runtime input regex

15.2 Architecture



15.3 Entities assumed to exist

Entity	Identifier	Notes
Services	`orders_api`, `orders_worker`, `orders_web`	Variables: `replicas`, `logLevel`.
Environments	`staging`, `prod_eu`, `prod_us`	Env variables: `ddEnv`, `expectedClusterName`, `region`.
Infrastructure	`staging_cluster`, `prod_eu_cluster`, `prod_us_cluster`	`allowSimultaneousDeployments: false`.
Connectors	`registry_main`, `github_main`, `servicenow_prod`, `datadog_main`, `k8s_*`	Delegate selectors set per tier.
Monitored services	One per service and environment (or a template)	Datadog health sources attached.
Overrides	`prod_eu` and `prod_us` service overrides	Replicas, log level.

Entity	Identifier	Notes
Barrier	`regions_verified`	Declared in the pipeline.

15.4 The pipeline YAML

```yaml
pipeline:
  name: orders_release
  identifier: orders_release
  projectIdentifier: payments
  orgIdentifier: platform
  tags:
    team: orders
  variables:
    - name: changeNumber
      type: String
      value: <+input>.regex(^CHG[0-9]{7}$)
    - name: releaseName
      type: String
      value: <+input>
  flowControl:
    barriers:
      - name: regions_verified
        identifier: regions_verified
  stages:
    - stage:
        name: staging_all
        identifier: staging_all
        type: Deployment
        spec:
          deploymentType: Kubernetes
          services:
            values:
              - serviceRef: orders_api
                serviceInputs:
                  serviceDefinition:
                    type: Kubernetes
                    spec:
                      artifacts:
                        primary:
                          primaryArtifactRef: <+input>
                          sources: <+input>
              - serviceRef: orders_worker
                serviceInputs:
                  serviceDefinition:
                    type: Kubernetes
                    spec:
                      artifacts:
                        primary:
                          primaryArtifactRef: <+input>
                          sources: <+input>
              - serviceRef: orders_web
                serviceInputs:
                  serviceDefinition:
                    type: Kubernetes
                    spec:
                      artifacts:
                        primary:
                          primaryArtifactRef: <+input>
                          sources: <+input>
            metadata:
              parallel: true
          environment:
            environmentRef: staging
            deployToAll: false
            infrastructureDefinitions:
              - identifier: staging_cluster
```

```
      execution:
        steps:
          - step:
              type: K8sRollingDeploy
              name: Rollout
              identifier: rollout
              timeout: 10m
              spec:
                skipDryRun: false
                pruningEnabled: false
          - step:
              type: Verify
              name: Verify Staging
              identifier: verify_staging
              timeout: 45m
              spec:
                type: Rolling
                monitoredService:
                  type: Default
                  spec: {}
                spec:
                  sensitivity: MEDIUM
                  duration: 10m
                  failOnNoAnalysis: false
              failureStrategies:
                - onFailure:
                    errors:
                      - Verification
                    action:
                      type: StageRollback
        rollbackSteps:
          - step:
              type: K8sRollingRollback
              name: Rollback
              identifier: rollback
              timeout: 10m
              spec:
                pruningEnabled: false
      failureStrategies:
        - onFailure:
            errors:
              - Connectivity
              - DelegateProvisioning
            action:
              type: Retry
              spec:
                retryCount: 2
                retryIntervals:
                  - 1m
                  - 3m
                onRetryFailure:
                  action:
                    type: ManualIntervention
                    spec:
                      timeout: 30m
                      onTimeout:
                        action:
                          type: Abort
        - onFailure:
            errors:
              - AllErrors
            action:
              type: StageRollback
  - stage:
      name: change_gate
      identifier: change_gate
      type: Approval
      spec:
        execution:
```

```
        steps:
          - step:
              type: ServiceNowApproval
              name: Change Approved In Window
              identifier: change_approved_in_window
              timeout: 2d
              spec:
                connectorRef: servicenow_prod
                ticketType: change_request
                ticketNumber: <+pipeline.variables.changeNumber>
                retryInterval: 2m
                approvalCriteria:
                  type: KeyValues
                  spec:
                    matchAnyCondition: false
                    conditions:
                      - key: state
                        operator: equals
                        value: Implement
                rejectionCriteria:
                  type: KeyValues
                  spec:
                    matchAnyCondition: true
                    conditions:
                      - key: state
                        operator: equals
                        value: Canceled
                changeWindow:
                  startField: start_date
                  endField: end_date
    failureStrategies:
      - onFailure:
          errors:
            - AllErrors
          action:
            type: Abort
- stage:
    name: prod_regions
    identifier: prod_regions
    type: Deployment
    strategy:
      matrix:
        region: [eu, us]
    spec:
      deploymentType: Kubernetes
      services:
        values:
          - serviceRef: orders_api
            serviceInputs:
              serviceDefinition:
                type: Kubernetes
                spec:
                  artifacts:
                    primary:
                      primaryArtifactRef: <+input>
                      sources: <+input>
          - serviceRef: orders_worker
            serviceInputs:
              serviceDefinition:
                type: Kubernetes
                spec:
                  artifacts:
                    primary:
                      primaryArtifactRef: <+input>
                      sources: <+input>
          - serviceRef: orders_web
            serviceInputs:
              serviceDefinition:
                type: Kubernetes
```

```
            spec:
              artifacts:
                primary:
                  primaryArtifactRef: <+input>
                  sources: <+input>
      metadata:
        parallel: true
    environment:
      environmentRef: prod_<+matrix.region>
      deployToAll: false
      infrastructureDefinitions:
        - identifier: prod_<+matrix.region>_cluster
    execution:
      steps:
        - step:
            type: Queue
            name: Serialize Production
            identifier: serialize_production
            timeout: 1h
            spec:
              key: orders-prod-<+env.identifier>
              scope: Pipeline
        - step:
            type: ShellScript
            name: Cluster Guard
            identifier: cluster_guard
            timeout: 2m
            spec:
              shell: Bash
              onDelegate: true
              source:
                type: Inline
                spec:
                  script: |
                    expected="<+env.variables.expectedClusterName>"
                    actual=$(kubectl config current-context 2>/dev/null || echo unknown)
                    echo "expected=$expected actual=$actual"
                    test "$expected" = "$actual"
              environmentVariables: []
              outputVariables: []
        - stepGroup:
            name: Canary
            identifier: canary
            steps:
              - step:
                  type: K8sCanaryDeploy
                  name: Canary Deploy
                  identifier: canary_deploy
                  timeout: 10m
                  spec:
                    instanceSelection:
                      type: Percentage
                      spec:
                        percentage: 10
                    skipDryRun: false
              - step:
                  type: Verify
                  name: Verify Canary
                  identifier: verify_canary
                  timeout: 1h
                  spec:
                    type: Canary
                    monitoredService:
                      type: Default
                      spec: {}
                    spec:
                      sensitivity: HIGH
                      duration: 15m
                      deploymentTag: <+serviceConfig.artifacts.primary.tag>
```

```
                      failOnNoAnalysis: true
                  failureStrategies:
                    - onFailure:
                        errors:
                          - Verification
                        action:
                          type: StageRollback
              - step:
                  type: Barrier
                  name: All Regions Verified
                  identifier: all_regions_verified
                  timeout: 45m
                  spec:
                    barrierRef: regions_verified
              - step:
                  type: K8sCanaryDelete
                  name: Canary Delete
                  identifier: canary_delete
                  timeout: 10m
                  spec: {}
          - step:
              type: K8sRollingDeploy
              name: Promote
              identifier: promote
              timeout: 20m
              spec:
                skipDryRun: false
                pruningEnabled: false
          - step:
              type: Verify
              name: Verify Promoted
              identifier: verify_promoted
              timeout: 1h
              spec:
                type: Rolling
                monitoredService:
                  type: Default
                  spec: {}
                spec:
                  sensitivity: MEDIUM
                  duration: 10m
                  failOnNoAnalysis: true
              failureStrategies:
                - onFailure:
                    errors:
                      - Verification
                    action:
                      type: StageRollback
      rollbackSteps:
        - step:
            type: K8sCanaryDelete
            name: Rollback Canary Delete
            identifier: rollback_canary_delete
            timeout: 10m
            spec: {}
        - step:
            type: K8sRollingRollback
            name: Rollback Rolling
            identifier: rollback_rolling
            timeout: 20m
            spec:
              pruningEnabled: false
    failureStrategies:
      - onFailure:
          errors:
            - Connectivity
            - DelegateProvisioning
          action:
            type: ManualIntervention
```

```
              spec:
                timeout: 30m
                onTimeout:
                  action:
                    type: Abort
        - onFailure:
            errors:
              - AllErrors
            action:
              type: StageRollback
  - stage:
      name: close_change
      identifier: close_change
      type: Custom
      when:
        pipelineStatus: All
      spec:
        execution:
          steps:
            - step:
                type: ServiceNowUpdate
                name: Update Change
                identifier: update_change
                timeout: 5m
                spec:
                  connectorRef: servicenow_prod
                  ticketType: change_request
                  ticketNumber: <+pipeline.variables.changeNumber>
                  useServiceNowTemplate: false
                  fields:
                    - name: state
                      value: Review
                    - name: close_notes
                      value: "Pipeline <+pipeline.executionUrl> finished with status <+pipeline.stages.prod_regions.status>."
```

15.5 Walkthrough of the design decisions

15.5.1 Staging as a multi-service stage

All three services deploy in parallel into `staging`, with a Verify step per deployment. If any service fails verification, its own rollback runs. Because the later production stage depends on a clean staging stage, the pipeline's default `when` (`Success`) prevents production from starting after a failed staging deployment.

15.5.2 Change gate as its own stage

The approval is isolated in an `Approval` stage. It runs after staging succeeds, so the CAB sees evidence that staging was green. The change window logic keeps production deployments inside the window recorded on the ticket. A failed approval aborts the pipeline, so no production work starts.

15.5.3 Matrix for regions, services in parallel

The production stage lists the same three services as staging and supplies their artifact inputs from an input set, so the versions that were verified in staging are the versions that go to production. (Service propagation with `useFromStage` is convenient for single-service stages, but it does not apply to a stage that deploys several services, which is why this design repeats the service list and relies on an input set.) The matrix expands the stage per region, and `services.metadata.parallel: true` deploys the three services of a region together. Each matrix iteration therefore contains three service deployments.

15.5.4 Barrier semantics

Every service deployment in every region passes through `all_regions_verified` after its canary Verify, so the barrier has six participants (three services times two regions). None promotes until all six canaries are healthy. Because `maxConcurrency` is unset and service deployments run in parallel, all participants are live at the same time, which avoids the barrier deadlock described in Chapter 6. Confirm in a sandbox how your Harness version counts barrier participants across the expanded services and iterations before you depend on this in production. If you want a per-region barrier instead of a global one, split the stage into one stage per region and use two parallel stages.

15.5.5 Queue step

`Serialize Production` ensures that two pipeline runs do not interleave in the same production environment. It is placed first so a second run waits before touching anything. Infrastructure-level queueing (`allowSimultaneousDeployments: false`) provides a second layer of protection.

15.5.6 Rollback behavior

Failure point	What happens
Cluster guard fails	Stage fails, rollback has nothing to undo (no deploy happened), pipeline fails.
Canary Verify fails in `eu`	`eu` rolls back (canary deleted). `us` is waiting at the barrier and is released as failed. Its rollback deletes its canary. Both regions end on the old version.
Promote fails in one region	That region rolls back with `K8sRollingRollback`. The other region may already have promoted. Decide whether to use `PipelineRollback` for atomicity.
Verify Promoted fails	`StageRollback` for that region.
Connectivity problem	Manual intervention, then abort on timeout.

15.5.7 Closing the change record

The final stage uses `when: pipelineStatus: All`, so it executes after success or failure. The close notes carry the status of production deployment, so the ticket always has a conclusion.

15.6 Operational parameters per environment

Service variables and overrides keep the pipeline unchanged across environments.

Variable	Service default	`staging`	`prod_eu`	`prod_us`
`replicas` (`orders_api`)	2	2	6	8
`logLevel`	info	debug	warn	warn
`ddEnv` (env var)	n/a	staging	prod-eu	prod-us
`expectedClusterName` (env var)	n/a	staging-1	prod-eu-1	prod-us-1

15.7 Rehearsal plan before first production use

1. Run the pipeline with `staging` standing in for both production regions in a sandbox copy.
2. Force a Verify failure in one region and confirm the barrier releases the other.
3. Reject the ServiceNow change and confirm the pipeline aborts before production.
4. Run with a closed change window and confirm the failure message.
5. Kill the delegate mid-rollout and confirm retry and manual intervention behavior.
6. Run two instances concurrently and confirm the Queue step serializes them.
7. Confirm the ServiceNow ticket ends in the right state for success and for failure.

Appendix A: Quick References

A.1 Expression cheat sheet

Need	Expression
Pipeline run number	`<+pipeline.sequenceId>`
Pipeline execution link	`<+pipeline.executionUrl>`
Who triggered	`<+pipeline.triggeredBy.name>`, `<+pipeline.triggeredBy.email>`
Pipeline variable	`<+pipeline.variables.name>`
Stage variable	`<+stage.variables.name>`
Service name and identifier	`<+service.name>`, `<+service.identifier>`
Effective service variable	`<+serviceVariables.name>`
Environment	`<+env.name>`, `<+env.identifier>`, `<+env.type>`
Environment variable	`<+env.variables.name>`
Infrastructure	`<+infra.name>`, `<+infra.namespace>`, `<+infra.releaseName>`
Primary artifact	`<+artifact.image>`, `<+artifact.tag>`, `<+artifact.imagePath>`
Sidecar artifact	`<+artifacts.sidecars.id.image>`
Secret	`<+secrets.getValue("name")>`
Step output variable	`<+execution.steps.stepId.output.outputVariables.name>`
Matrix value	`<+matrix.axis>`
Repeat item and index	`<+repeat.item>`, `<+repeat.index>`
Iteration	`<+strategy.iteration>`, `<+strategy.iterations>`
Iteration suffix	`<+strategy.identifierPostFix>`
Ternary	`<+ <+env.type> == "Production" ? "a" : "b" >`
HTTP step response	`<+httpResponseCode>`, `<+httpResponseBody>`
Config file contents	`<+configFile.getAsString("id")>`
Account or project variable	`<+variable.account.name>`, `<+variable.name>`
Trigger artifact tag	`<+trigger.artifact.build>`

A.2 Runtime input modifiers

Modifier	Example
Plain	`<+input>`
Default	`<+input>.default(qa)`
Allowed values	`<+input>.allowedValues(dev,qa,prod)`
Regex	`<+input>.regex(^CHG[0-9]{7}$)`
Execution input	`<+input>.executionInput()`
Combined	`<+input>.default(qa).allowedValues(dev,qa,prod)`

A.3 Step type reference (Kubernetes CD)

Step type	Role
`K8sRollingDeploy`	Rolling deploy.
`K8sRollingRollback`	Rolling rollback.
`K8sCanaryDeploy`	Create canary workload.
`K8sCanaryDelete`	Remove canary workload.
`K8sBlueGreenDeploy`	Create stage workload and Services.
`K8sBGSwapServices`	Swap blue-green Services.
`K8sBlueGreenStageScaleDown`	Scale down after swap.
`K8sApply`	Apply manifests as-is.
`K8sDelete`	Delete resources.
`K8sScale`	Scale a workload.
`K8sDryRun`	Validate manifests.
`Verify`	Continuous verification.
`Barrier`	Synchronize parallel branches.
`Queue`	Serialize via resource key.
`HarnessApproval`	Native approval.
`ServiceNowCreate`, `ServiceNowUpdate`, `ServiceNowApproval`, `ServiceNowImportSet`	ServiceNow integration.
`ShellScript`	Run a script.
`Http`	Call an HTTP endpoint.

A.4 Failure strategy quick table

Situation	Suggested action
Application failure during rollout	`StageRollback`
Verification failure (mature)	`StageRollback`
Verification failure (early adoption)	`ManualIntervention`, rollback on timeout
Connectivity or delegate problem	`Retry`, then `ManualIntervention`
Approval rejected	`Abort`
Cleanup stages	`when: pipelineStatus: All`
Non-critical notification step	`Ignore` or `MarkAsSuccess`
Multi-stage atomic release	`PipelineRollback`

A.5 Override precedence summary

Priority	Source
Lowest	Service definition
	Environment global override
	Environment and service override
Highest	Infrastructure and service override

Confirm in a sandbox with a Probe step (Chapter 4.7).

Appendix B: Production Readiness Checklist

B.1 Pipeline structure

Check	
Services contain no environment-specific literals.	
Artifact tag is runtime input or expression, with digest pinning where needed.	
Environment variables hold region, domain, and cluster identity.	
Overrides are used for per-environment tuning.	
Every step has an explicit timeout.	
Pipelines, templates, and input sets are stored in Git.	

B.2 Safety

Check	
Cluster guard step in production stages.	
Dry run enabled in production.	
Queue or infrastructure queueing prevents concurrent releases.	
Freeze windows configured for known events.	
RBAC separates authors from approvers.	
Secrets are stored in an external manager.	

B.3 Change management

Check	
ServiceNow approval stage precedes production.	
Change window enforced.	
Ticket updated on both success and failure.	
Change number validated with regex.	
No retries on ServiceNow create steps.	

B.4 Verification

Check	
Unified Service Tagging on workloads.	
Datadog queries grouped by Pod.	
Fail-fast thresholds for critical metrics.	
`failOnNoAnalysis: true` in production when data is reliable.	
Failure strategy for `Verification` defined.	
Tested with a deliberately bad build.	

B.5 Rollback

Check	
`rollbackSteps` present in every deployment stage.	
Failure strategy triggers rollback.	
Versioned ConfigMaps and Secrets.	
Release history permissions verified.	
Image retention covers rollback targets.	
Emergency rollback pipeline rehearsed.	
Blue-green scale-down delayed until after observation.	

B.6 Operations

Check	
Delegates sized for peak parallelism.	
Delegate selectors reflect environment tier.	
Notifications for failures, approvals, and rollbacks.	
Pipeline failure rate and delegate health monitored.	
Runbook includes the manual deployment path.	

Appendix C: Glossary

Term	Definition
Artifact source	A registry or repository location from which a service takes its deployable image or package.
Barrier	A pipeline-level synchronization point for parallel branches.
Canary	Deploying a small slice of the new version alongside the stable one.
Change window	The period in which a ServiceNow change is authorized to run.
Connector	Stored configuration and credentials for an external system.
Delegate	The Harness worker in your network that executes tasks.
Environment	A logical deployment target such as `prod_eu`.
Environment group	A named collection of environments.
Expression	A `<+...>` reference resolved at run time.
Failure strategy	Rules mapping error types to actions.
Freeze window	A period during which deployments are blocked.
Health source	A telemetry source attached to a monitored service.
Infrastructure definition	The concrete target (cluster, namespace) inside an environment.
Input set	A saved set of runtime input values.
Managed workload	The workload Harness tracks for steady state and rollback.
Matrix	A looping strategy over combinations of values.
Monitored service	The entity that links a service and environment to health sources.
Overlay input set	A composition of input sets applied in order.
Release history	The in-cluster record of what Harness applied per release.
Repeat	A looping strategy over a list of items.
Resource constraint	A mechanism that limits concurrent use of a resource.
Service	The definition of a deployable workload.
Service variable	A named value on a service, overridable per environment.
Step group	A set of steps sharing strategy, rollback, and conditions.
Steady state	A workload condition where it is fully rolled out and healthy.
Verify step	The continuous verification step.

Appendix D: Validation and Further Reading

D.1 Validate this material against your account

The platform evolves. Before standardizing patterns from this guide:

1. Paste each YAML sample into the Pipeline Studio YAML editor in a sandbox project. The editor validates against the live schema.
2. For connectors, services, environments, and infrastructure, create one through the UI and compare the generated YAML to the samples.
3. For features with notes in this guide (override model, filters, partition syntax, post-deployment rollback, JEXL accessors), run a small experiment before relying on them.
4. If you use the Harness MCP server in your editor, you can fetch the exact schema for an entity before creating it, which reduces trial and error.

D.2 Topics outside the scope of this guide

Topic	Note
GitOps (Argo CD based)	A separate delivery model with its own sync and rollback semantics.
Native Helm, ECS, serverless, and VM deployment types	The concepts of flow control, verification, approvals, and governance transfer, but steps differ.
Feature flags and progressive delivery	Pair with deployment strategies for finer-grained exposure control.
Chaos engineering	Complements verification by testing failure behavior on purpose.
Pipeline v1 (simplified YAML)	Same ideas, different syntax (`${{ }}` expressions).